STRONG QUANTITATIVE TYPING CHARACTERIZES STRONG NORMALIZATION AND PERPETUAL EVALUATION IN THE FUNCTIONAL MACHINE CALCULUS

WILLEM HEIJLTJES

Editorial note. Source https://arxiv.org/abs/2505.09960v1. Copyright Willem Heijltjes, CC BY4.0 for this fixed extended article. Conference version DOI10.4230/LIPIcs.FSCD.2025.24 is recorded as the same work. Theorem29 is counted once, and the complete native proof dependencies in appendicesB-F are included. Bundled proof-dashed.sty remains GPL1-or-later under its own copyright notice.

Allowed standard prerequisites (editorial).

- First-order binding/substitution, natural-number well-founded induction and non-idempotent intersection types. Extended article sections2-6. Actual FMC-specific typing, reduction, termination and perpetual-evaluation proofs are retained, not deferred to background.

2. The Functional Machine Calculus

This section will define the FMC: its syntax, the abstract machine, a quantitative big-step semantics, its reduction relations, and the type system.

The primary intuition for the FMC is as an instruction language for a machine with multiple stacks, indexed by a set of *locations* $\{a, b, c, \dots\}$, and together called a *memory*. The calculus interacts with individual stacks through *push* and *pop* constructs: $[N]a.\, M$ pushes the term N to the stack a and continues as M, and $a\langle x\rangle.\, M$ pops from the stack a, using the popped term as x in M. This generalizes the evaluation of λ-terms on the (simplified) Krivine machine [30], where application pushes its argument, and abstraction pops. The standard, call–by–name λ-calculus [40] then embeds by choosing a *default* location λ, by letting $M\,N = [N]\lambda.\, M$ and $\lambda x.M = \lambda\langle x\rangle.\, M$. For brevity, the location λ may be omitted, writing instead $[N].\, M$ respectively $\langle x\rangle.\, M$. The locations of the FMC may encode *reader/writer* effects: higher order store as stacks of depth one, I/O as separate input and output streams, and non-determinism and probabilistic choice as non-deterministically and probabilistically generated input streams (see [24] for details).

The FMC further includes *sequencing* $M\,;N$ and its unit *skip* $\star$. These are composition and identity of FMC terms viewed as *memory transformers*: computations with an input memory and output memory. The big-step semantics, below, uses this interpretation directly, while it is implemented on the machine via a continuation stack, in standard fashion.

Definition 1 (Syntax)**.** *The* terms *of the Functional Machine Calculus are given by the following grammar.*

$$M,\, N,\, P \;::=\; x \;\mid\; a\langle x\rangle.\, M \;\mid\; [N]a.\, M \;\mid\; \star \;\mid\; M\,;N$$

Date: arxiv:2505.09960v1; CC BY4.0 (arXiv2505.09960v1 extended article); bundled proof-dashed.sty GPL1.0-or-later.

From left to right these are: a variable x, *an* abstraction *or* pop $a\langle x\rangle.\, M$ *on location* a *which binds* x *in* M, *an* application *or* push $[N]a.\, M$ *of term* N *on location* a, *the* unit *or* skip $\star$, *and* sequencing $M\,;N$.

The operational meaning of the five constructs will be expressed through a small-step semantics, in the form of a simple stack machine, and a big-step semantics, in the form of an inductive evaluation relation. The abstract machine will use the following components. An *operand stack* or *argument stack* S is a stack of terms, represented with the head on the right and using ε for the empty stack. A *memory* is a family of operand stacks indexed in a set of locations A. A *continuation stack* is a stack of terms with the head to the left.

$$S, T \;::=\; \varepsilon \;\mid\; S\,M \qquad S_A \;::=\; \{S_a \mid a \in A\} \qquad K, L \;::=\; M\,K \;\mid\; \varepsilon$$

For a memory S_A, the stack a being empty, ε_a, or outside the family, $a \notin A$, are equivalent. Thus we may fix a set of locations A going forward, noting that any given term uses only a fixed, finite set of locations, determined syntactically. Let $S_A \cdot M_a$ denote the memory S_A with M pushed to the stack indexed by a.

Definition 2 (Abstract machine)**.** *A* machine state (S_A, M, K) *is a triple of a memory* S_A, *a term* M, *and a continuation stack* K. *The* transitions *or* steps *of the machine given by the following rules.*

$$\frac{(\; S_A \;,\; [N]a.\, M \;,\; K \;)}{(\; S_A \cdot N_a \;,\; M \;,\; K \;)} \qquad \frac{(\; S_A \;,\; M\,;N \;,\; K \;)}{(\; S_A \;,\; M \;,\; N\,K \;)}$$

$$\frac{(\; S_A \cdot N_a \;,\; a\langle x\rangle.\, M \;,\; K \;)}{(\; S_A \;,\; \{N/x\}M \;,\; K \;)} \qquad \frac{(\; S_A \;,\; \star \;,\; M\,K \;)}{(\; S_A \;,\; M \;,\; K \;)}$$

A final state *is of the form* $(S_A, \star, \varepsilon)$ *and a* failure state *of either of the forms* (S_A, x, K) *or* $(S_A, a\langle x\rangle.\, M, K)$ *where* $S_a = \varepsilon$. *A* run *is a sequence of steps, written as below, where the* length *of the run* $n \geq 1$ *is the number of states it traverses (which may be omitted from the notation). A run is* successful *if it terminates in a final state.*

$$\frac{(\; S_A \;,\; M \;,\; K \;)}{\;\;\;

Proposition 4 (Small-step and big-step semantics agree)**.** *Successful machine runs of length n correspond to quantified evaluation $\Downarrow_n$:*

$$\frac{(\ S_A\ ,\ M\ ,\ \varepsilon\ )}{(\ T_A\ ,\ \star\ ,\ \varepsilon\ )}n \quad\Longleftrightarrow\quad S_A,\ M \Downarrow_n T_A$$

We will see two reduction relations. Regular reduction $\rightarrow$ is a rewrite relation, closed under all contexts, and spine reduction $\rightarrow_{\mathsf{w}}$ is restricted to spine contexts, which means everywhere except within the argument N of an application $[N]a.\,M$.

Definition 5 (Reduction)**.** *The* reduction rules *are as follows.*

Beta:	$[N]a.\,a\langle x\rangle.\,M$	$\rightarrow$	$\{N/x\}M$	
Passage:	$[N]b.\,a\langle x\rangle.\,M$	$\rightarrow$	$a\langle x\rangle.\,[N]b.\,M$	$(a \neq b, x \notin \mathsf{fv}(N))$
Next:	$\star\,;M$	$\rightarrow$	M	
Prefix (pop):	$a\langle x\rangle.\,N\,;M$	$\rightarrow$	$a\langle x\rangle.\,(N\,;M)$	$(x \notin \mathsf{fv}(M))$
Prefix (push):	$[P]a.\,N\,;M$	$\rightarrow$	$[P]a.\,(N\,;M)$	
Associate:	$(P\,;N)\,;M$	$\rightarrow$	$P\,;(N\,;M)$	

Reduction contexts C *are given by the following grammar.*

$$C \ ::=\ \{\} \ \mid\ a\langle x\rangle.\,C \ \mid\ [M]a.\,C \ \mid\ [C]a.\,M \ \mid\ C\,;M \ \mid\ M\,;C$$

Spine contexts W *are given by the following grammar.*

$$W \ :=\ \{\} \ \mid\ a\langle x\rangle.\,W \ \mid\ [M]a.\,W \ \mid\ W\,;M \ \mid\ M\,;W$$

The (capturing) substitution of the hole $\{\}$ in a context C by a term M is written $C\{M\}$. The reduction *relation $\rightarrow$ is given by closing the reduction rules $M \rightarrow N$ under any context, $C\{M\} \rightarrow C\{N\}$.* Spine *reduction $\rightarrow_{\mathsf{w}}$ is given by closing under spine contexts, $W\{M\} \rightarrow_{\mathsf{w}} W\{N\}$. The reflexive-transitive closure of a relation $\rightarrow$ is written $\twoheadrightarrow$, and reduction to normal form as $\twoheadrightarrow\!\!\!\!|$.*

Without the beta-rule, reduction is strongly normalizing.

Proposition 6. *Non-beta reduction is SN.*

Proof. The following pair of measures strictly decreases. Writing $|M|$ for the size of a term M, first the sum over $|M|$ for every subterm $M\,;N$ (equal for *passage*, decreasing for other rules); second the sum over $|M|$ for every subterm $[N]a.\,M$ (decreasing for *passage*). □

The above definitions differ from the original presentation [24] in the following ways, and for the following reasons. First, instead of introducing sequencing $M\,;N$ as a primitive, the original syntax used a sequential variable $x.\,M$ and defined sequencing as an operation. The two formulations are equivalent modulo the reductions for sequencing (the Next, Prefix, and Associate reductions), whose normal forms use sequencing exclusively as $x\,;M$. The original formulation reduced the number of reduction rules and avoided the need for a continuation stack on the machine, emphasizing the view of terms as sequences of machine instructions. Present thinking however prefers simplicity not through fewer but through more elementary components, viewing the more refined reduction relation and the continuation stack as benefits rather than obstacles. The current syntax emphasizes the clean combination of λ-calculus and sequencing and more readily accommodates current and future extensions such as exceptions [25] and concurrency.

Second, the original formulation used a single yet complex reduction rule, below, where the ellipsis represents any number of abstractions and applications on locations other than a.

$$[N]a\ldots a\langle x\rangle.\,M \rightarrow \ldots\{N/x\}M$$

Again the emphasis has shifted to multiple simpler rules, Beta and Passage. These follow the natural case analysis for an application meeting an abstraction, whether they have the same or different locations, and give more structured normal forms, where all abstractions precede all applications, as we will see in Sections 4 and 6.

The main technical results of the paper, in the following sections, will establish the relation between the operational semantics, reduction, and quantitative types. The latter are defined next.

Definition 7 (Quantitative types)**.** Quantitative types *for the FMC are built up in four mutually recursive layers :* computation types*,* collection types*,* vector types*, and* memory types*, given by the following grammars.*

$$\begin{array}{lll} \textit{Computation types:} & & \tau ::= \overline{\iota} \Rightarrow \overline{\overline{\kappa}} \\ \textit{Collection types:} & \iota,\, \kappa,\, \mu,\, \nu,\, \pi & ::= [\tau_1, \ldots, \tau_n] \\ \textit{Vector types:} & & \overline{\iota} ::= \iota_1 \ldots \iota_n \\ \textit{Memory types:} & & \overline{\overline{\iota}} ::= \{\overline{\iota}_a \mid a \in A\} \end{array}$$

Collection types are multisets, i.e. considered modulo associativity, symmetry, and unitality, where multiset union is denoted as a sum, $\iota + \kappa$*.*

These four categories reflect the components of the operational semantics. Terms will be typed with computation types (or simply *types*) $M : \tau$, as will continuation stacks $K : \tau$. Collection types may be viewed as *value types*, along the computation/value distinction of call–by–push–value [32], and apply to the argument term in an application and to the terms on an argument stack (note the continuation stack holds terms as *computations*). Argument stacks themselves are typed by vector types, $S : \overline{\iota}$, and memories by memory types, $S_A : \overline{\overline{\iota}}$.

Note that the base cases for the definition are given by empty collection types $[]$ and empty vector and memory types, both denoted ε. Composition of type vectors is by juxtaposition, $\overline{\iota}\,\overline{\kappa}$, extended to memories point-wise, $\overline{\overline{\iota}}\,\overline{\overline{\kappa}} = \{\overline{\iota}_a\,\overline{\kappa}_a \mid a \in A\}$. The singleton family $a(\overline{\iota})$ has $\overline{\iota}$ at location a and ε at any other location. The default location λ may again be omitted, writing $\overline{\iota}$ for $\lambda(\overline{\iota})$.

Since stacks are first-in-first-out, identity terms are of the form below, with pops and pushes in inverse order to each other. To match this in the type system, type vectors on the left of an implication are implicitly reversed, so that identity types may be written $\overline{\overline{\iota}} \Rightarrow \overline{\overline{\iota}}$.

$$a_n\langle x_n\rangle \ldots a_1\langle x_1\rangle.\, [x_1]a_1 \ldots [x_n]a_n.\star \;:\; a_n(\iota_n) \ldots a_1(\iota_1) \Rightarrow a_1(\iota_1) \ldots a_n(\iota_n)$$

A *typing context* Γ, Δ, Λ, Σ is a function from variables to collection types, written as a sequence $\Gamma = x_1 : \iota_1, \ldots, x_n : \iota_n$ where any other variable y is implicitly assigned the empty type, $\Gamma(y) = []$. The *union* or *sum* of two contexts $\Gamma + \Delta$ is given pointwise: $(\Gamma + \Delta)(x) = \Gamma(x) + \Delta(x)$.

Definition 8 (The quantitatively typed FMC)**.** *The typing rules in Figure 1 define the* quantitatively-typed Functional Machine Calculus*, in two variants:* weak *and* strong*, with typing judgements* $\Gamma \vdash_{\mathsf{w}} M : \tau$ *respectively* $\Gamma \vdash_{\mathsf{s}} M : \tau$ *that assign the term* M *the type* τ *in the context* Γ*. The weak type system is given by the rules of Figure 1 with unlabelled* ($\vdash$) *and weak judgements* ($\vdash_{\mathsf{w}}$)*, and the strong system by those with unlabelled* ($\vdash$) *and strong judgements* ($\vdash_{\mathsf{s}}$)*. The* weight n *of a derivation is the number of rule instances for abstraction, application, sequencing, and unit, and may be indicated in a judgement as* $\vdash^n$ *.*

The first section of Figure 1 gives the weak system for *terms*, the next section extends the weak system to memories, continuation stacks, and machine states, and the final section gives the specific rules for the strong type system (which is defined only for terms, not states). When it is clear from the context whether a judgement

Variable: $$\frac{}{x\colon [\tau] \vdash x\colon \tau}\,\mathsf{var}$$

Abstraction: $$\frac{\Gamma,\, x\colon \iota \vdash M\colon \overline{\kappa} \Rightarrow \overline{\mu}}{\Gamma \vdash a\langle x\rangle.\, M\colon a(\iota)\, \overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{abs}$$

Application (weak): $$\frac{\Gamma \vdash_{\mathsf{w}} N\colon \iota \qquad \Delta \vdash_{\mathsf{w}} M\colon a(\iota)\, \overline{\kappa} \Rightarrow \overline{\mu}}{\Gamma + \Delta \vdash_{\mathsf{w}} [N]a.\, M\colon \overline{\iota} \Rightarrow \overline{\mu}}\,\mathsf{app}$$

Unit: $$\frac{}{\vdash \star\colon \overline{\iota} \Rightarrow \overline{\iota}}\,\mathsf{unit}$$

Sequencing: $$\frac{\Gamma \vdash N\colon \overline{\iota} \Rightarrow \overline{\kappa} \qquad \Delta \vdash M\colon \overline{\kappa} \Rightarrow \overline{\mu}}{\Gamma + \Delta \vdash N\,;M\colon \overline{\iota} \Rightarrow \overline{\mu}}\,\mathsf{seq}$$

Collection: $$\frac{(\Gamma_i \vdash M\colon \tau_i)_{1 \le i \le n}}{\Gamma_1 + \ldots + \Gamma_n \vdash M\colon [\tau_1, \ldots, \tau_n]}\,\mathsf{col}$$

Memory: $$\frac{}{\vdash \varepsilon_A\colon \varepsilon}\,\mathsf{m0} \qquad \frac{\vdash S_A\colon \overline{\iota} \qquad \vdash M\colon \kappa}{\vdash S_A \cdot M_a\colon \overline{\iota}\, a(\kappa)}\,\mathsf{m1}$$

Continuation stack: $$\frac{}{\vdash \varepsilon\colon \overline{\iota} \Rightarrow \overline{\iota}}\,\mathsf{k0} \qquad \frac{\Gamma \vdash M\colon \overline{\iota} \Rightarrow \overline{\kappa} \qquad \vdash K\colon \overline{\kappa} \Rightarrow \overline{\mu}}{\vdash M\,K\colon \overline{\iota} \Rightarrow \overline{\mu}}\,\mathsf{k1}$$

State: $$\frac{\vdash S_A\colon \overline{\iota} \qquad \vdash M\colon \overline{\iota} \Rightarrow \overline{\kappa} \qquad \vdash K\colon \overline{\kappa} \Rightarrow \overline{\mu}}{\vdash (S_A, M, K)\colon \varepsilon \Rightarrow \overline{\mu}}\,\mathsf{st}$$

Application (strong): $$\frac{\Gamma \vdash_{\mathsf{s}} N\colon \iota \qquad \Delta \vdash_{\mathsf{s}} M\colon a(\iota)\, \overline{\kappa} \Rightarrow \overline{\mu} \qquad \Lambda \vdash_{\mathsf{s}} N\colon \tau}{\Gamma + \Delta + \Lambda \vdash_{\mathsf{s}} [N]a.\, M\colon \overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{APP}$$

Weakening: $$\frac{\Gamma \vdash_{\mathsf{s}} M\colon \tau}{\Gamma + \Delta \vdash_{\mathsf{s}} M\colon \tau}\,\mathsf{WEAK}$$

FIGURE 1. The quantitatively typed Functional Machine Calculus

concerns the weak or the strong type system, the subscripts on $\vdash_{\mathsf{w}}$ and $\vdash_{\mathsf{s}}$ may be dropped.

The key difference between both systems is in the third premise to the strong application rule. In both application rules, the first premise assigns a type to the argument N which may be empty in the case where it will not be used by M. This corresponds to machine evaluation, where N may be discarded without evaluation, but not to strong normalization, where N must itself be normalizing. The strong application rule therefore requires N to be typed separately, a standard approach [8, 11], here via the third premise.

Typing judgements for machine states are of the form $\vdash (S_A, M, K)\colon \varepsilon \Rightarrow \overline{\iota}$, with empty context and input type. This prevents failure states from being typed: the empty context means there are no computation types for free variables, and an empty input type means there must always be sufficient items in memory for any abstraction to pop. *Progress*, the idea that a machine state is either final or has a transition, is thus immediate from types, once it is proved that machine transitions preserve typing in Lemma 10 below.

Non-idempotent intersection types for the λ-calculus [11] embed into quantitative FMC types in the manner of simple types [24], by the following interpretation, using the default location λ (omitted from the notation). The weak type system then embeds the system $\mathcal{W}$ for λ-terms of [11], while the strong system differs from system $\mathcal{S}$ in two minor ways: in the presence of the weakening rule, and for the strong application rule, in requiring a witness also when the argument has a non-empty collection type.

$$\begin{array}{ll} \text{Base type:} & o = \varepsilon \Rightarrow \varepsilon \\ \text{Arrow types:} & [\sigma_1, \dots, \sigma_n] \to \tau = [\sigma_1, \dots, \sigma_n]\, \overline{\kappa} \Rightarrow \overline{\mu} \quad \text{where } \tau = \overline{\kappa} \Rightarrow \overline{\mu} \end{array}$$

3. Machine Termination

This section will show that the successful evaluation and weak typeability coincide, where the weights agree. First, in the weak type system, the following *substitution* typing rule is admissible: it can be introduced without altering the expressiveness of the type system.

$$\frac{\Gamma \vdash N : \iota \qquad \Delta, x : \iota \vdash M : \tau}{\Gamma + \Delta \vdash \{N/x\}M : \tau}\,\mathsf{sub}$$

Lemma 9 (Substitution, weak)**.** *There are derivations* $\Gamma \Vdash^n_{\mathsf{w}} N : \nu$ *and* $\Delta, x : \nu \Vdash^m_{\mathsf{w}} M : \tau$ *if and only if there is a derivation* $\Gamma + \Delta \Vdash^k_{\mathsf{w}} \{N/x\}M : \tau$ *where* $k = n + m$*.*

For the weakly typed calculus, a machine step from a typed state leads to a typed state of weight exactly one less.

Lemma 10 (Evaluation reduces typing weights)**.** *If*

$$\frac{(\ S_A\ ,\ M\ ,\ K\ )}{(\ T_A\ ,\ N\ ,\ L\ )}$$

then $\Vdash^n_{\mathsf{w}} (S_A, M, K) : \varepsilon \Rightarrow \overline{\iota}$ *if and only if* $\Vdash^{n-1}_{\mathsf{w}} (T_A, N, L) : \varepsilon \Rightarrow \overline{\iota}$*.*

It follows that weak types guarantee termination of the machine while measuring the length of a run.

Theorem 11 (Weak types quantify evaluation)**.** *For a typed state* $\Vdash^n_{\mathsf{w}} (S_A, M, \varepsilon) : \varepsilon \Rightarrow \overline{\overline{\iota}}$ *there exists a typed memory* $\Vdash^m_{\mathsf{w}} T_A : \overline{\iota}$ *such that* $S_A,\ M \Downarrow_k T_A$ *where* $n = m + k$*.*

Proof. Recall that failure states cannot be typed. A typed state $\Vdash^n_{\mathsf{w}} (S_A, M, K) : \varepsilon \Rightarrow \overline{\iota}$ is then either final or has a machine transition. In the case of a transition, by Lemma 10 the new state is typed with weight one less. Thus, the machine continues taking steps, preserving types but reducing weights, until it reaches a final state, giving a run of length k as below left, so that $S_A,\ M \Downarrow_k T_A$ by Proposition 4. The final state $(T_A, \star, \varepsilon)$ is typed as below right, with weight $m+1$ where $\Vdash^m_{\mathsf{w}} T_A : \overline{\iota}$. This represents a run of length 1, so that a run of length k starts from a weight $m + k = n$.

$$\frac{(\ S_A\ ,\ M\ ,\ \varepsilon\ )}{(\ T_A\ ,\ \star\ ,\ \varepsilon\ )}\,k \qquad \frac{\Vdash^m_{\mathsf{w}} T_A : \overline{\iota} \qquad \overline{\Vdash^1_{\mathsf{w}} \star : \overline{\iota} \Rightarrow \overline{\iota}} \qquad \overline{\Vdash^0_{\mathsf{w}} \varepsilon : \overline{\iota} \Rightarrow \overline{\iota}}}{\Vdash^{m+1}_{\mathsf{w}} (T_A, \star, \varepsilon) : \varepsilon \Rightarrow \overline{\iota}}\,\mathsf{st} \qquad \square$$

For the reverse, that all terminating states can be typed, it is now sufficient to give a type to final states $(T_A, \star, \varepsilon)$. We may do so simply by typing each term in the memory T_A with the empty collection type $[]$. Denote by $[]^n$ the type vector of $[] \dots []$ of length n, and by $[]^f$ for a function $f : A \to \mathbb{N}$ the memory type $\{[]^{f(a)} \mid a \in A\}$. Let $|T|$ denote the length of a stack T and let $|T_A| = f$ where $f(a) = |T_a|$ denote the dimensions of a memory T_A. For types, likewise let $|\overline{\iota}|$ denote the length of a type vector and $|\overline{\overline{\iota}}| = f$ where $f(a) = |\overline{\iota}_a|$ the dimensions of a memory type.

Theorem 12 (Evaluation implies weak typing)**.** *If* $S_A,\ M \Downarrow_n T_A$ *then* $\vdash^n_{\mathsf{w}} (S_A, M, \varepsilon) \colon \varepsilon \Rightarrow []^f$ *where* $f = |T_A|$.

Proof. The memory T_A may be typed $\vdash^0_{\mathsf{w}} T_A \colon []^f$, and its final state $\vdash^0_{\mathsf{w}} (T_A, \star, \varepsilon) \colon \varepsilon \Rightarrow []^f$. By Proposition 4 there is a run of length n to this state from (S_A, M, ε). By Lemma 10 each step in this run, backwards from the final state, gives a previous state with the same type but weight one greater, so that by induction $\vdash^n_{\mathsf{w}} (S_A, M, \varepsilon) \colon \varepsilon \Rightarrow []^f$. □

Combining both theorems, since a typeable state $\vdash_{\mathsf{w}} (S_A, M, \varepsilon) \colon \varepsilon \Rightarrow \overline{\iota}$ terminates with a memory $T_A : \overline{\iota}$, which is also typed by $[]^f$ where $f = |\overline{\iota}|$, we have $\vdash^n_{\mathsf{w}} (S_A, M, \varepsilon) \colon \varepsilon \Rightarrow []^f$ where the weight n of the latter is exactly the number of machine steps to successful termination.

4. Spine Normalization

The present section will establish that weak typing and successful evaluation further coincide with spine normalization. First we observe the shape of spine normal forms and demonstrate that they may be typed.

Proposition 13 (Spine normal forms)**.** *The normal forms of spine reduction* $\rightarrowtriangle_{\mathsf{w}}$ *are the terms* $\mathcal{W}$ *given by the following mutually recursive grammars.*

$$\begin{aligned} \mathcal{W} &::= a\langle x\rangle.\, \mathcal{W} \mid \mathcal{V} \\ \mathcal{V} &::= [M]a.\, \mathcal{V} \mid x\,;\mathcal{W} \mid x \mid \star \end{aligned}$$

Lemma 14 (Spine normal forms are weakly typeable)**.** *For any* $\mathcal{W}$ *there is a derivation* $\Gamma \vdash_{\mathsf{w}} \mathcal{W} \colon \overline{\iota} \Rightarrow []^f$.

Spine reduction preserves and reflects typing, where weights are strictly reduced in the reduction rules *beta* ($[N]a.\, a\langle x\rangle.\, M \longrightarrow \{N/x\}M$) and *next* ($\star\,; M \longrightarrow M$), and preserved otherwise.

Lemma 15 (Weighted subject reduction and expansion)**.** *If* $M \rightarrowtriangle_{\mathsf{w}} N$ *then* $\Gamma \vdash^m_{\mathsf{w}} M \colon \tau$ *if and only if* $\Gamma \vdash^n_{\mathsf{w}} N \colon \tau$*, where* $m = n + 2$ *in the case of a beta-step or next-step and* $m = n$ *otherwise.*

It now follows that weak typeability and spine normalization coincide.

Theorem 16 (Weak typing characterizes spine normalization)**.** *A term is weakly typeable if and only if it is spine-normalizing.*

Proof. From left to right, since non-beta steps are strongly normalizing by Proposition 6, an infinite reduction must contain infinitely many beta-steps. By Lemma 15 spine beta-reduction decreases the weight of the typing derivation, while other steps do not increase it. Then spine reduction is terminating. From right to left, proceed by induction on the sum length of all spine reduction paths. Spine-normal terms are typeable by Lemma 14, and for a step $M \rightarrowtriangle_{\mathsf{w}} N$ if N is typeable then so is M by Lemma 15. □

Since weak typing characterizes both spine normalization and machine termination (by Theorems 11 and 12), the natural corollary would be that machine termination and spine normalization coincide. In one direction, this follows directly: by Theorem 12 termination implies typeability, and by Theorem 16 typeability implies spine normalization.

In the other direction, however, there is still a small gap to be closed: a spine-normalizing term M may be typeable as $\Gamma \vdash_{\mathsf{w}} M \colon \overline{\iota} \Rightarrow \overline{\overline{\kappa}}$, but the machine is only guaranteed to run for states of type $\varepsilon \Rightarrow \overline{\overline{\kappa}}$. In other words, for M to run successfully, a memory S_A of type $\vdash_{\mathsf{w}} S_A \colon \overline{\iota}$ must be provided, and each variable $x \colon \iota$ in Γ must

be substituted by a term N typed $\vdash_{\mathsf{w}} N : \iota$. However, these are not guaranteed to exist, since not all quantitative types are inhabited. This is in contrast with simple types for the FMC, which *are* inhabited [24, Remark 3.7].

Proposition 17 (No type inhabitation)**.** *There exist types τ such that $\not\vdash^{\mathsf{w}} M : \tau$ for any M.*

Proof. Since types are preserved under spine reduction, if a type is inhabited by a term, it is inhabited by its spine normal form. The only normal form inhabiting the type $\varepsilon \Rightarrow \varepsilon$ is $\vdash \star : \varepsilon \Rightarrow \varepsilon$ (it cannot be an abstraction, since the type has no inputs; it cannot contain free variables, since the context is empty; and it cannot be an application, since the type has no outputs). Similarly, the only spine normal forms inhabiting $\varepsilon \Rightarrow [\varepsilon \Rightarrow \varepsilon]$ are of the form $\vdash [M].\star : \varepsilon \Rightarrow [\varepsilon \Rightarrow \varepsilon]$. It follows that the collection type $[\varepsilon \Rightarrow \varepsilon, \varepsilon \Rightarrow [\varepsilon \Rightarrow \varepsilon]]$ is not inhabited, and so neither is the type $\varepsilon \Rightarrow [\varepsilon \Rightarrow \varepsilon, \varepsilon \Rightarrow [\varepsilon \Rightarrow \varepsilon]]$. □

To connect spine normalization and termination, for every spine normal term $\mathcal{W}$ a memory S_A and substitution map σ will be given such that for S_A and $\sigma\mathcal{W}$ the machine terminates. A *substitution map* $\sigma = \{M_1/x_1, \ldots, M_n/x_n\}$ is a finite function from variables to terms, applied as σM to a term M as a simultaneous substitution. To allow a successful machine run for $\sigma\mathcal{W}$, the memory S_A and substitution map σ must be such that, at any point in the computation, there are sufficient terms on all stacks to pop from. To this end the following notions are defined: for a natural number d, a *memory of dimension d* will have at least d terms of dimension $d-1$ on each stack, and a *term of dimension d* consists of sufficient applications to generate a memory of dimension d. These are formalized as follows, again noting that the locations A for a given term are finite and may be determined syntactically.

For a stack $S = \varepsilon\, N_1 \ldots N_n$ let $[S]a.\, M$ denote the term $[N_1]a \ldots [N_n]a.\, M$, and for a memory S_A let $[S_A].\, M$ denote a term $[S_{a_1}]a_1 \ldots [S_{a_n}]a_n.\, M$ where $A = \{a_1 \ldots a_n\}$. For a natural number d, a *memory of dimension d* is one S_A where $|S_A| = f$ with $f(a) \geq d$ for all $a \in A$, and each term in S_A is a term of dimension $d-1$, where a *term of dimension d* is of the form $[S_A].\star$ with S_A of dimension d. A *substitution map of dimension d* is one that sends every variable to a term of dimension d. Observe that an element of dimension d is also one of any $d' < d$.

Lemma 18 (Spine normal forms terminate)**.** *For a spine-normal term $\mathcal{W}$ there is a dimension d such that for any memory S_A and substitution map σ of dimension d, there is a memory T_A such that $S_A, \sigma\mathcal{W} \Downarrow T_A$.*

Since machine termination guarantees typing (Theorem 12), the above would be sufficient to connect spine normalization to weak typeability. The following lemma is however instructive in connecting spine reduction and machine termination explicitly.

Lemma 19 (Evaluation is invariant under spine reduction)**.** *If $M \twoheadrightarrow_{\mathsf{w}} N$ then $S_A, \sigma M \Downarrow T_A$ if and only if $S_A, \sigma N \Downarrow T_A$ for any memories S_A and T_A and substitution map σ.*

The main theorem connecting weak typing, spine normalization, and machine termination is then the following.

Theorem 20 (Weak types characterize termination and spine normalization)**.** *For a term M the following are equivalent:*

(1) *M is weakly typeable $\Gamma \vdash_{\mathsf{w}} M : \tau$,*
(2) *M is spine-normalizing,*

(3) M *is machine-terminating: there are memories* S_A, T_A *and a substitution map* σ *such that* $S_A, \sigma M \Downarrow T_A$.

Proof. By Theorem 16 statements 1 and 2 are equivalent. From statement 2 to 3, if M spine normalizes to $\mathcal{W}$ then Lemma 18 gives memories S_A and T_A and a substitution map σ such that $S_A, \sigma\mathcal{W} \Downarrow T_A$, and since machine evaluation is reflected by spine reduction (Lemma 19), also $S_A, \sigma M \Downarrow T_A$. From 3 to 1, by Theorem 12 for $S_A, \sigma M \Downarrow T_A$ we get $\vdash_{\mathsf{w}} (S_A, \sigma M, \varepsilon) \colon \varepsilon \Rightarrow []^f$ where $f = |T_A|$. The typing rule for states gives $\vdash_{\mathsf{w}} S_A \colon \overline{\iota}$ and $\vdash_{\mathsf{w}} \sigma M \colon \overline{\iota} \Rightarrow []^f$ for some $\overline{\iota}$. By the weak substitution lemma (Lemma 9), since σM is typeable so is M, and we have $\Gamma \vdash_{\mathsf{w}} M \colon \overline{\iota} \Rightarrow []^f$ for some Γ. □

Note that the above theorem resolves the problem of uninhabited types as follows: if M is typed $\Gamma \vdash_{\mathsf{w}} M \colon \tau$ where either Γ or τ contains an uninhabited (input) type, then (by $1 \Rightarrow 2$) it has a spine normal form, which in turn (by $2 \Rightarrow 3$) evaluates on the machine, which then (by $3 \Rightarrow 1$) guarantees that M may also be typed with only inhabited types.

5. Strong Normalization

Where the weak type system characterizes machine evaluation and spine normalization, the strong type system characterizes strong normalization (SN). This section will prove one direction, that normalization reduces the weight of strong type derivations and, hence, that strong quantitative types guarantee SN and give upper bounds to the length of reductions.

We start with the substitution lemma for strong types. The substitution typing rule can be eliminated, maintaining or reducing weight, and introduced, maintaining weight. The inequality is due to the weakening rule WEAK: substitution into a weakened part of the context erases the typing information of a term, maintaining only the types for its context.

Lemma 21 (Substitution, strong)**.** *If there are derivations* $\Gamma \vdash^n_{\mathsf{s}} N \colon \nu$ *and* $x \colon \nu$, $\Delta \vdash^m_{\mathsf{s}} M \colon \tau$ *then there is a derivation* $\Gamma + \Delta \vdash^k_{\mathsf{s}} \{N/x\}M \colon \tau$ *where* $k \leq n + m$*, and vice versa where* $k = n + m$.

The following lemma gives preservation of typing for beta-reduction, reducing weights.

Lemma 22 (Subject reduction, beta)**.** *If* $M \twoheadrightarrow N$ *by a beta-step and* $\Gamma \vdash^m_{\mathsf{s}} M \colon \tau$ *then* $\Gamma \vdash^n_{\mathsf{s}} N \colon \tau$ *where* $m > n$.

While we have subject reduction, note that we do not have subject *expansion*. In the reduction $[N]a.\, a\langle x\rangle.\, M \twoheadrightarrow \{N/x\}M$, if x is not free in M, there is no typing information for N in the reduct, but this is required for the strong application rule in the redex. This familiar situation presents an obstacle in demonstrating that all strongly normalising terms can be typed, covered in the next section.

For the remaining reduction rules, we do have subject expansion as well as reduction. Recall that the *next* reduction, $\star\,; M \twoheadrightarrow M$, reduces weights, while the other non-beta reduction rules maintain weights. It would be possible to use an adjusted weight that counts *only* application and abstraction typing rules, to give exact weights for non-beta subject reduction, since non-beta rules are inherently terminating by Proposition 6.

Lemma 23 (Subject reduction and expansion, non-beta)**.** *If* $M \twoheadrightarrow N$ *by a non-beta-step then* $\Gamma \vdash^m_{\mathsf{s}} M \colon \tau$ *if and only if* $\Gamma \vdash^n_{\mathsf{s}} N \colon \tau$ *where* $m \geq n$.

We conclude this section with the theorem that typed terms are strongly normalizing.

Theorem 24 (Strong normalization)**.** *If* $\Gamma \vdash_{\mathsf{s}} M : \tau$ *then* M *is strongly normalizing.*

Proof. By *subject reduction*, Lemmata 22 and 23, reduction preserves types while maintaining or reducing weights. Since beta-steps strictly reduce the weight of a derivation and non-beta-steps are strongly normalizing by Proposition 6, all reduction paths must be finite. □

6. Perpetual Evaluation

For the reverse direction, that all strongly normalizing terms are typeable, the challenge is that of *weakened* terms, the argument terms N in a reduction $[N]a.\, a\langle x\rangle.\, M \longrightarrow \{N/x\}M$ where x is not free in M. Reasoning by beta-expansion, to derive SN for $[N]a.\, a\langle x\rangle.\, M$ we need termination information for N, which the reduct $\{N/x\}M = M$ does not immediately provide. This problem constitutes the essential difference between the λI- and λK-calculus (see [3]) and is familiar from many strong normalization arguments, in particular those deriving SN from weak normalization by maintaining the weakened term N in the term structure [29, 35] and those using perpetual reduction strategies [6, 43].

The present approach will be a variant of perpetual reduction. We will see a *perpetual evaluation* relation $M \blacktriangleright \mathcal{M}$ which relates a term M to its normal form $\mathcal{M}$, that in the case of a redex $[N]a.\, a\langle x\rangle.\, M$ not only evaluates $\{N/x\}M$ but in addition requires an evaluation $N \blacktriangleright \mathcal{M}'$. This definition, as a big-step evaluation relation instead of a strategy, avoids maintaining weakened terms in the syntax, either as redexes or explicit substitutions.

The proof then becomes straightforward: strong normalizability of M implies perpetual evaluation $M \blacktriangleright \mathcal{M}$, implies strong typeability $\Gamma \vdash_{\mathsf{s}} M : \tau$. As before, we begin by observing the shape of normal forms.

Proposition 25 (Normal forms)**.** *The normal forms of* $\longrightarrow$ *are the terms* $\mathcal{M}$ *given by the following grammars.*

$$\begin{aligned} \mathcal{M} &::= a\langle x\rangle.\, \mathcal{M} \mid \mathcal{N} \\ \mathcal{N} &::= [\mathcal{M}]a.\, \mathcal{N} \mid x\,;\mathcal{M} \mid x \mid \star \end{aligned}$$

Perpetual evaluation will reduce terms to normal form by *iterated weak head reduction*. That is, terms are reduced in a context of applications, a *weak head context*, which functions similarly to the memory of the abstract machine. Where the machine would reach a *final* or *failure* state, reduction continues (i.e. is *iterated*) for the remaining subterms, which on the machine would constitute the memory and continuation stack.

We will denote a term M in weak head context $[N_1]a_1 \ldots [N_n]a_n.\, M$ as $[S_A].\, M$, where $S_A = \varepsilon\cdot(N_1)_{a_1}\cdots(N_n)_{a_n}$ is the memory corresponding to the sequence of applications $[N_1]a_1$ through $[N_n]a_n$. Note that the notation $[S_A]$ is not a function on S_A, since multiple application sequences may correspond to the same memory (to be exact: those modulo $[P]b.\,[N]a.\, M \sim [N]a.\,[P]b.\, M$ where $a \neq b$). Typing, however, collapses these distinctions again: a term $[S_A].\star$ will have the type $\varepsilon \Rightarrow \overline{\overline{\iota}}$ where $S_A : \overline{\overline{\iota}}$.

Definition 26 (Perpetual evaluation)**.** *The* perpetual evaluation *relation* $M \blacktriangleright \mathcal{M}$ *between terms and normal forms is given inductively by the rules in Figure 2.*

The salient features of perpetual evaluation are the following. The first six rules each correspond to a reduction rule in weak head context. Simultaneously, these can be seen to implement machine evaluation, where the context represents the memory. The beta-rule requires the additional normalization of the argument N, as discussed above. The remaining four rules cover four of the five cases of normal forms, with the fifth, for application, covered by the presence of weak head contexts. As

$$\text{Beta}\quad \frac{[S_A].\,\{N/x\}M \blacktriangleright \mathcal{M} \qquad N \blacktriangleright \mathcal{M}'}{[S_A].\,[N]a.\,a\langle x\rangle.\,M \blacktriangleright \mathcal{M}} \qquad \frac{[S_A].\,a\langle x\rangle.\,[N]b.\,M \blacktriangleright \mathcal{M}}{[S_A].\,[N]b.\,a\langle x\rangle.\,M \blacktriangleright \mathcal{M}}\quad \text{Passage}$$

$$\text{Next}\quad \frac{[S_A].\,M \blacktriangleright \mathcal{M}}{[S_A].\,(\star\,;M) \blacktriangleright \mathcal{M}} \qquad \frac{[S_A].\,a\langle x\rangle.\,(N\,;M) \blacktriangleright \mathcal{M}}{[S_A].\,(a\langle x\rangle.\,N\,;M) \blacktriangleright \mathcal{M}}\quad \text{Prefix (pop)}$$

$$\text{Associate}\quad \frac{[S_A].\,(P\,;(N\,;M)) \blacktriangleright \mathcal{M}}{[S_A].\,((P\,;N)\,;M) \blacktriangleright \mathcal{M}} \qquad \frac{[S_A].\,[P]a.\,(N\,;M) \blacktriangleright \mathcal{M}}{[S_A].\,([P]a.\,N\,;M) \blacktriangleright \mathcal{M}}\quad \text{Prefix (push)}$$

$$\text{Normal (abstraction)}\quad \frac{M \blacktriangleright \mathcal{M}}{a\langle x\rangle.\,M \blacktriangleright a\langle x\rangle.\,\mathcal{M}}$$

$$\text{Normal (unit)}\quad \frac{(N_i \blacktriangleright \mathcal{M}_i)_{i\leq n}}{[N_1]a_1\ldots[N_n]a_n.\star \blacktriangleright [\mathcal{M}_1]a_1\ldots[\mathcal{M}_n]a_n.\star}$$

$$\text{Normal (variable)}\quad \frac{(N_i \blacktriangleright \mathcal{M}_i)_{i\leq n}}{[N_1]a_1\ldots[N_n]a_n.x \blacktriangleright [\mathcal{M}_1]a_1\ldots[\mathcal{M}_n]a_n.x}$$

$$\text{Normal (sequence)}\quad \frac{(N_i \blacktriangleright \mathcal{M}_i)_{i\leq n} \qquad M \blacktriangleright \mathcal{M}}{[N_1]a_1\ldots[N_n]a_n.(x\,;M) \blacktriangleright [\mathcal{M}_1]a_1\ldots[\mathcal{M}_n]a_n.(x\,;\mathcal{M})}$$

FIGURE 2. Perpetual evaluation

discussed above, these cases represent final or failure states on the machine, where reduction needs to iterate on the subterms representing the remaining memory and continuation stack.

The following lemma then shows that a strongly normalizing term also has a finite perpetual evaluation. To avoid reference to confluence, it is phrased as a combination of two statements: one, that SN implies perpetual evaluation, and two, that the normal form given by perpetual evaluation is indeed a normal form of the original term.

Lemma 27 (Strong normalization implies perpetual evaluation)**.** *If M is strongly normalizing then $M \blacktriangleright \mathcal{M}$ and $M \twoheadrightarrow\!\!\!\!\blacktriangleright \mathcal{M}$ for some $\mathcal{M}$.*

The definition of perpetual evaluation as an inductive relation means we may assign the same type to a term and its normal form simultaneously.

Lemma 28 (Perpetual evaluation implies strong typeability)**.** *If $M \blacktriangleright \mathcal{M}$ then $\Gamma \vdash_{\mathsf{s}} M : \tau$ and $\Gamma \vdash_{\mathsf{s}} \mathcal{M} : \tau$ for some Γ and τ.*

This gives the required equivalence between strong quantitative typing, strong normalization, and perpetual evaluation.

Theorem 29 (Strong typing characterizes strong normalization)**.** *For a term M, the following are equivalent:*

(1) *M is strongly typeable, $\Gamma \vdash_{\mathsf{s}} M : \tau$,*
(2) *M is strongly normalizing,*
(3) *there is a perpetual evaluation $M \blacktriangleright \mathcal{M}$.*

Proof. From 1 to 2 is Theorem 24, from 2 to 3 is Lemma 27, and from 3 to 1 is Lemma 28. □

Appendix B. Proofs for Section 2: The Functional Machine Calculus

Proposition 30 (Small-step and big-step semantics agree)**.** *Successful machine runs of length n correspond to quantified evaluation $\Downarrow_n$:*

$$\frac{(\ S_A\ ,\ M\ ,\ \varepsilon\ )}{(\ T_A\ ,\ \star\ ,\ \varepsilon\ )}n \quad\Longleftrightarrow\quad S_A,\ M \Downarrow_n T_A$$

Proof. The machine transitions correspond to the big-step evaluation rules as follows. From left to right the proof is by (strong) induction on n, and from right to left by induction on the derivation for $\Downarrow_n$.

- **Application:** A successful run starting with a *push* transition and the evaluation rule for application correspond as follows.

$$\frac{\dfrac{(\ S_A\ ,\ [N]a.\,M\ ,\ \varepsilon\ )}{(\ S_A\cdot N_a\ ,\ M\ ,\ \varepsilon\ )}}{(\ T_A\ ,\ \star\ ,\ \varepsilon\ )}n \qquad \frac{S_A\cdot N_a,\ M \Downarrow_n T_A}{S_A,\ [N]a.\,M \Downarrow_{n+1} T_A}$$

- **Abstraction:** A successful run starting with a *pop* transition and the evaluation rule for abstraction correspond as follows.

$$\frac{\dfrac{(\ S_A\cdot N_a\ ,\ a\langle x\rangle.\,M\ ,\ \varepsilon\ )}{(\ S_A\ ,\ M\ ,\ \varepsilon\ )}}{(\ T_A\ ,\ \star\ ,\ \varepsilon\ )}n \qquad \frac{S_A,\ \{N/x\}M \Downarrow_n T_A}{S_A\cdot N_a,\ a\langle x\rangle.\,M \Downarrow_{n+1} T_A}$$

- **Unit:** The unit run for a single state corresponds to the unit evaluation rule.

$$(S_A, \star, \varepsilon) \qquad \frac{}{S_A,\ \star \Downarrow_1 S_A}$$

- **Sequence:** A successful run for $M\,;N$ consists of a transition pushing N to the continuation stack, a successful run for M (ignoring N on the continuation stack), a transition popping N, and a run for N. The correspondence is then as follows, where the total length of the run is that of both constituting runs for M and N plus the initial state for $M\,;N$.

$$\frac{\dfrac{\dfrac{(\ R_A\ ,\ M\,;N\ ,\ \varepsilon\ )}{(\ R_A\ ,\ M\ ,\ N\ )}}{(\ S_A\ ,\ \star\ ,\ N\ )}m}{\dfrac{(\ S_A\ ,\ N\ ,\ \varepsilon\ )}{(\ T_A\ ,\ \star\ ,\ \varepsilon\ )}}n \qquad \frac{R_A,\ M \Downarrow_m S_A \qquad S_A,\ N \Downarrow_n T_A}{R_A,\ M\,;N \Downarrow_{m+n+1} T_A}$$

□

Appendix C. Proofs for Section 3: Machine Termination

Lemma 31 (Substitution, weak)**.** *There are derivations $\Gamma \vdash^n_{\mathsf{w}} N : \nu$ and $\Delta,\ x : \nu \vdash^m_{\mathsf{w}} M : \tau$ if and only if there is a derivation $\Gamma + \Delta \vdash^k_{\mathsf{w}} \{N/x\}M : \tau$ where $k = n + m$.*

Proof. The proof is by induction on M, except that the *collection* case is treated separately from the *application* case. Each case is demonstrated by two derivations, representing a permutation of the substitution typing rule, where one derivation exists if and only if the other does.

- **Variable case:** In the case $M = x$ we have the following two derivations. In both directions it is immedate that given one derivation, we may

construct the other.

$$\dfrac{\dfrac{\Gamma \vdash N : \tau}{\Gamma \vdash N : [\tau]}\,\mathsf{col} \qquad \dfrac{}{x : [\tau] \vdash x : \tau}\,\mathsf{var}}{\Gamma \vdash \{N/x\}x : \tau}\,\mathsf{sub} \qquad\qquad \Gamma \vdash N : \tau$$

In the case $M = y \neq x$ we have the following two derivations.

$$\dfrac{\dfrac{}{\vdash N : []}\,\mathsf{col} \qquad \dfrac{}{y : [\tau], x : [] \vdash y : \tau}\,\mathsf{var}}{y : [\tau] \vdash \{N/x\}y : \tau}\,\mathsf{sub} \qquad\qquad \dfrac{}{y : [\tau] \vdash y : \tau}\,\mathsf{var}$$

- **Abstraction case:** In the case $M = a\langle y\rangle.\, P$ we have the following two derivations. In both directions, given one derivation we may construct the other, where by α-conversion we may assume $y \notin \mathsf{fv}(N)$.

$$\dfrac{\Gamma \vdash N : \nu \qquad \dfrac{\dfrac{}{\Delta, x : \nu, y : \iota, \vdash P : \overline{\kappa} \Rightarrow \overline{\mu}}}{\Delta, x : \nu \vdash \langle y\rangle.\, P : a(\iota)\,\overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{abs}}{\Gamma + \Delta \vdash \{N/x\}a\langle y\rangle.\, P : a(\iota)\,\overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{sub}$$

$$\dfrac{\dfrac{\Gamma \vdash N : \nu \qquad \Delta, x : \nu, y : \iota, \vdash P : \overline{\kappa} \Rightarrow \overline{\mu}}{\Gamma + \Delta, y : \iota \vdash \{N/x\}P : \overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{sub}}{\Gamma + \Delta \vdash a\langle y\rangle.\, \{N/x\}P : a(\iota)\,\overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{abs}$$

- **Weak application case:** In the case $M = [Q]a.\, P$ we have the following two derivations, where $\nu = \nu_1 + \nu_2$ and $\Delta = \Delta_1 + \Delta_2$. Given the former derivation, since $\Gamma \vdash N : \nu$ must be introduced by a collection rule, we have $\Gamma_i \vdash N : \nu_i$ for $i = 1, 2$ with $\Gamma = \Gamma_1 + \Gamma_2$, so that we may construct the latter derivation. The other direction is immediate.

$$\dfrac{\Gamma \vdash N : \nu \qquad \dfrac{\Delta_1, x : \nu_1 \vdash Q : \iota \qquad \Delta_2, x : \nu_2 \vdash P : a(\iota)\,\overline{\kappa} \Rightarrow \mu}{\Delta, x : \nu \vdash_{\mathsf{w}} [Q]a.\, P}\,\mathsf{app}}{\Gamma + \Delta \vdash \{N/x\}[Q]a.\, P : \overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{sub}$$

$$\dfrac{\dfrac{\Gamma_1 \vdash N : \nu_1 \qquad \Delta_1, x : \nu_1 \vdash Q : \iota}{\Gamma_1 + \Delta_1 \vdash \{N/x\}Q : \iota}\,\mathsf{sub} \qquad \dfrac{\Gamma_2 \vdash N : \nu_2 \qquad \Delta_2, x : \nu_2 \vdash P : a(\iota)\,\overline{\kappa} \Rightarrow \overline{\mu}}{\Gamma_2 + \Delta_2 \vdash \{N/x\}P : a(\iota)\,\overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{sub}}{\Gamma + \Delta \vdash_{\mathsf{w}} [\{N/x\}Q]a.\, \{N/x\}P : \overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{app}$$

- **Unit case:** We have the following two derivations.

$$\dfrac{\dfrac{}{\vdash N : []}\,\mathsf{col} \qquad \dfrac{}{\vdash \star : \overline{\iota} \Rightarrow \overline{\iota}}\,\mathsf{unit}}{\vdash \{N/x\}\star : \overline{\iota} \Rightarrow \overline{\iota}}\,\mathsf{sub} \qquad\qquad \dfrac{}{\vdash \star : \overline{\iota} \Rightarrow \overline{\iota}}\,\mathsf{unit}$$

- **Sequencing case:** The case is similar to the weak application case, with the following two derivations. Given the former derivation, since $\nu = \nu_1 + \nu_2$ and $\Gamma \vdash N : \nu$ is introduced by an *intersection* rule, we have the two derivations $\Gamma_i \vdash N : \nu_i$ for $i = 1, 2$, and may construct the latter derivation. The reverse is again immediate, noting that $\Gamma = \Gamma_1 + \Gamma_2$ and $\Delta = \Delta_1 + \Delta_2$.

$$\dfrac{\Gamma \vdash N : \nu \qquad \dfrac{\Delta_1, x : \nu_1 \vdash P : \overline{\iota} \Rightarrow \overline{\kappa} \qquad \Delta_2, x : \nu_2 \vdash Q : \overline{\kappa} \Rightarrow \overline{\mu}}{\Delta, x : \nu \vdash P\,;Q : \overline{\iota} \Rightarrow \overline{\mu}}\,\mathsf{seq}}{\Gamma + \Delta \vdash \{N/x\}(P\,;Q) : \overline{\iota} \Rightarrow \overline{\mu}}\,\mathsf{sub}$$

$$\dfrac{\dfrac{\Gamma_1 \vdash N : \nu_1 \qquad \Delta_1, x : \nu_1 \vdash P : \overline{\iota} \Rightarrow \overline{\kappa}}{\Gamma_1 + \Delta_1 \vdash \{N/x\}P : \overline{\iota} \Rightarrow \overline{\kappa}}\,\mathsf{sub} \qquad \dfrac{\Gamma_2 \vdash N : \nu_2 \qquad \Delta_2, x : \nu_2 \vdash Q : \overline{\kappa} \Rightarrow \overline{\mu}}{\Gamma_2 + \Delta_2 \vdash \{N/x\}Q : \overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{sub}}{\Gamma + \Delta \vdash \{N/x\}(P\,;Q) : \overline{\iota} \Rightarrow \overline{\mu}}\,\mathsf{seq}$$

- **Collection case:** The case is similar to the weak application case. The two derivations are as follows, where $\nu = \nu_1 + \ldots + \nu_n$ and $\Delta = \Delta_1 + \ldots + \Delta_n$. Given the former derivation, since $\Gamma \vdash N : \nu$ is introduced by an intersection rule, we have the derivations $\Gamma_i \vdash N : \nu_i$ for $1 \leq i \leq n$ needed to construct the latter derivation, where $\Gamma = \Gamma_1 + \ldots + \Gamma_n$. The reverse direction is immediate.

$$\dfrac{\Gamma \vdash N : \nu \qquad \dfrac{(\Delta_i,\, x : \nu_i \vdash M : \tau_i)_{1 \leq i \leq n}}{\Delta,\, x : \nu \vdash M : [\tau_1, \ldots, \tau_n]}\,\mathsf{col}}{\Gamma + \Delta \vdash \{N/x\}M : [\tau_1, \ldots, \tau_n]}\,\mathsf{sub}$$

$$\dfrac{\left(\dfrac{\Gamma_i \vdash N : \nu_i \qquad \Delta_i,\, x : \nu_i \vdash M : \tau_i}{\Gamma_i + \Delta_i \vdash \{N/x\}M : \tau_i}\,\mathsf{sub}\right)_{1 \leq i \leq n}}{\Gamma + \Delta \vdash \{N/x\}M : [\tau_1, \ldots, \tau_n]}\,\mathsf{col}$$

□

Lemma 32 (Evaluation reduces typing weights)**.** *If*

$$\frac{(\ S_A\ ,\ M\ ,\ K\ )}{(\ T_A\ ,\ N\ ,\ L\ )}$$

then $\vdash^n_{\mathsf{w}} (S_A, M, K) : \varepsilon \Rightarrow \overline{\iota}$ *if and only if* $\vdash^{n-1}_{\mathsf{w}} (T_A, N, L) : \varepsilon \Rightarrow \overline{\iota}$.

Proof. For each of the four machine transitions the two typing derivations will be given. In each case, given one derivation the other may be constructed, while the first has weight exactly one greater than the second, as it has exactly one additional rule instance contributing to the weight of a derivation (an *application*, *abstraction*, *unit*, or *sequencing* rule).

- **Application case:**

$$\frac{(\ S_A\ ,\ [N]a.\, M\ ,\ K\ )}{(\ S_A \cdot N_a\ ,\ M\ ,\ K\ )}$$

The two typing derivations are as follows.

$$\dfrac{\vdash S_A : \overline{\iota} \qquad \dfrac{\vdash N : \nu \qquad \vdash M : a(\nu)\,\overline{\iota} \Rightarrow \overline{\overline{\kappa}}}{\vdash [N]a.\, M : \overline{\iota} \Rightarrow \overline{\overline{\kappa}}}\,\mathsf{app} \qquad \vdash K : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}}{\vdash (S_A, [N]a.\, M, K) : \varepsilon \Rightarrow \overline{\overline{\mu}}}\,\mathsf{st}$$

$$\dfrac{\dfrac{\vdash S_A : \overline{\iota} \qquad \vdash N : \nu}{\vdash S_A \cdot N_a : \overline{\iota}\, a(\nu)}\,\mathsf{m1} \qquad \vdash M : a(\nu)\,\overline{\iota} \Rightarrow \overline{\overline{\kappa}} \qquad \vdash K : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}}{\vdash (S_A \cdot N_a, M, K) : \varepsilon \Rightarrow \overline{\overline{\mu}}}\,\mathsf{st}$$

- **Abstraction case:**

$$\frac{(\ S_A \cdot N_a\ ,\ \ a\langle x\rangle.\, M\ ,\ K\ )}{(\ S_A\ \ ,\ \{N/x\}M\ ,\ K\ )}$$

The two typing derivations are as follows. In both directions, the use of the *substitution* typing rule is justified by Lemma 9.

$$\dfrac{\dfrac{\vdash S_A : \overline{\iota} \qquad \vdash N : \nu}{\vdash S_A \cdot N_a : \overline{\iota}\, a(\nu)}\,\mathsf{m1} \qquad \dfrac{x : \nu \vdash M : \overline{\iota} \Rightarrow \overline{\overline{\kappa}}}{\vdash a\langle x\rangle.\, M : a(\nu)\,\overline{\iota} \Rightarrow \overline{\overline{\kappa}}}\,\mathsf{abs} \qquad \vdash K : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}}{\vdash (S_A \cdot N_a, a\langle x\rangle.\, M, K) : \varepsilon \Rightarrow \overline{\overline{\mu}}}\,\mathsf{st}$$

$$\dfrac{\vdash S_A : \overline{\iota} \qquad \dfrac{\vdash N : \nu \qquad x : \nu \vdash M : \overline{\iota} \Rightarrow \overline{\overline{\kappa}}}{\vdash \{N/x\}M : \overline{\iota} \Rightarrow \overline{\overline{\kappa}}}\,\mathsf{sub} \qquad \vdash K : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}}{\vdash (S_A, \{N/x\}M, K) : \varepsilon \Rightarrow \overline{\overline{\mu}}}\,\mathsf{st}$$

- **Sequencing case:**

$$\frac{(\ S_A\ ,\ M\,;N\ ,\quad K\ )}{(\ S_A\ ,\quad M\ ,\ N\,K\ )}$$

The two derivations are as follows.

$$\dfrac{\vdash S_A : \overline{\iota} \qquad \dfrac{\vdash M : \overline{\iota} \Rightarrow \overline{\kappa} \qquad \vdash N : \overline{\kappa} \Rightarrow \overline{\mu}}{\vdash M\,;N : \overline{\iota} \Rightarrow \overline{\mu}}\,\mathsf{seq} \qquad \vdash K : \overline{\mu} \Rightarrow \overline{\nu}}{\vdash (S_A, M\,;N, K) : \varepsilon \Rightarrow \overline{\nu}}\,\mathsf{st}$$

$$\dfrac{\vdash S_A : \overline{\iota} \qquad \vdash M : \overline{\iota} \Rightarrow \overline{\kappa} \qquad \dfrac{\vdash N : \overline{\kappa} \Rightarrow \overline{\mu} \qquad \vdash K : \overline{\mu} \Rightarrow \overline{\nu}}{\vdash N\,K : \overline{\kappa} \Rightarrow \overline{\nu}}\,\mathsf{k1}}{\vdash (S_A, M, N\,K) : \varepsilon \Rightarrow \overline{\nu}}\,\mathsf{st}$$

- **Unit case:**

$$\frac{(\ S_A\ ,\quad \star\ ,\ M\,K\ )}{(\ S_A\ ,\ M\ ,\quad K\ )}$$

The two derivations are as follows.

$$\dfrac{\vdash S_A : \overline{\iota} \qquad \dfrac{}{\vdash \star : \overline{\iota} \Rightarrow \overline{\iota}}\,\mathsf{unit} \qquad \dfrac{\vdash M : \overline{\iota} \Rightarrow \overline{\kappa} \qquad \vdash K : \overline{\kappa} \Rightarrow \overline{\mu}}{\vdash M\,K : \overline{\iota} \Rightarrow \overline{\mu}}\,\mathsf{k1}}{\vdash (S_A, \star, M\,K) : \varepsilon \Rightarrow \overline{\mu}}\,\mathsf{st}$$

$$\dfrac{\vdash S_A : \overline{\iota} \qquad \vdash M : \overline{\iota} \Rightarrow \overline{\kappa} \qquad \vdash K : \overline{\kappa} \Rightarrow \overline{\mu}}{\vdash (S_A, M, K) : \varepsilon \Rightarrow \overline{\mu}}\,\mathsf{st}$$

□

Appendix D. Proofs for Section 4: Spine Normalization

Proposition 33 (Spine normal forms)**.** *The normal forms of spine reduction* $\twoheadrightarrow_{\mathsf{w}}$ *are the terms* $\mathcal{W}$ *given by the following mutually recursive grammars.*

$$\begin{array}{rcl} \mathcal{W} & ::= & a\langle x\rangle.\,\mathcal{W} \mid \mathcal{V} \\ \mathcal{V} & ::= & [M]a.\,\mathcal{V} \mid x\,;\mathcal{W} \mid x \mid \star \end{array}$$

Proof. By induction on terms, it is shown that $\mathcal{W}$ are the spine normal terms, and $\mathcal{V}$ those that are not abstractions. A variable x and a unit $\star$ are spine normal, and a composition $M\,;N$ is normal if and only if M is a variable and N is spine normal, since if M is a unit, application, abstraction, or sequencing, a reduction step applies. An application $[N]a.\,M$ is spine-normal if and only if M is spine-normal and not an abstraction, since otherwise a beta-step or passage-step would apply. An abstraction $a\langle x\rangle.\,M$ is spine-normal if and ony if M is. □

Lemma 34 (Spine normal forms are weakly typeable)**.** *For any* $\mathcal{W}$ *there is a derivation* $\Gamma \vdash_{\mathsf{w}} \mathcal{W} : \overline{\iota} \Rightarrow []^f$.

Proof. By induction on $\mathcal{W}$ the following are proved.

(1) For any $\mathcal{W}$ there is a type derivation $\Gamma \vdash_{\mathsf{w}} \mathcal{W} : \overline{\iota} \Rightarrow []^f$.
(2) For any $\mathcal{V}$ and any $f : A \to \mathbb{N}$ there is a type derivation $\Gamma \vdash_{\mathsf{w}} \mathcal{V} : []^f \Rightarrow []^g$.

Note that 2 implies 1.

- **Unit case:** For a term $\star$, we prove case 2. For any f we have $\vdash \star : []^f \Rightarrow []^f$.
- **Variable case:** For a variable x, we prove case 2. For any f we have $x : [[]^f \Rightarrow \varepsilon] \vdash x : []^f \Rightarrow \varepsilon$.
- **Sequencing case:** For a term $x\,;\mathcal{W}$ we prove case 2. By induction, let $\Gamma \vdash \mathcal{W} : \overline{\iota} \Rightarrow []^g$. For any f we get $\Gamma + (x : []^f \Rightarrow \overline{\iota}) \vdash x\,;\mathcal{W} : []^f \Rightarrow []^g$.

- **Application case:** For a term $[M]a.\,\mathcal{V}$, we prove case 2. Given f, define f' by $f'(a) = f(a) + 1$ and $f'(b) = f(b)$ for $a \neq b$. We apply the inductive hypothesis for case 2 to $\mathcal{V}$ and f' to get $\Gamma \vdash \mathcal{V} \colon []^{f'} \Rightarrow []^{g}$. The following *weak application* rule instance then gives the required type.

$$\dfrac{\dfrac{}{\vdash M \colon []}\,\mathsf{col} \qquad \Gamma \vdash \mathcal{V} \colon []^{f'} \Rightarrow []^{g}}{\Gamma \vdash_{\mathsf{w}} [M]a.\,\mathcal{V} \colon []^{f} \Rightarrow []^{g}}\,\mathsf{app}$$

- **Abstraction case:** For a term $a\langle x\rangle.\,\mathcal{W}$ we prove case 1. By induction, there is a judgement $\Gamma,\, x \colon \iota \vdash \mathcal{W} \colon \overline{\kappa} \Rightarrow []^{f}$. Then by the *abstraction* typing rule we get the required type $\Gamma \vdash a\langle x\rangle.\,\mathcal{W} \colon a(\iota)\,\overline{\kappa} \Rightarrow []^{f}$. □

Lemma 35 (Weighted subject reduction and expansion)**.** *If $M \twoheadrightarrow_{\mathsf{w}} N$ then $\Gamma \vdash^{m}_{\mathsf{w}} M \colon \tau$ if and only if $\Gamma \vdash^{n}_{\mathsf{w}} N \colon \tau$, where $m = n + 2$ in the case of a beta-step or next-step and $m = n$ otherwise.*

Proof. By induction on spine contexts. The cases for a top-level reduction step are as follows.

- **Beta case:** For a reduction $[N]a.\,a\langle x\rangle.\,M \longrightarrow \{N/x\}M$, by Lemma 9 a derivation for $\{N/x\}M$ is given by a substitution rule, as below right. Then we have the derivation below left if and only if we have that below right, with weight $n + 2$ respectively n.

$$\dfrac{\Gamma \vdash N \colon \iota \qquad \dfrac{\dfrac{\Delta,\, x \colon \iota \vdash M \colon \overline{\kappa} \Rightarrow \overline{\mu}}{\Delta \vdash a\langle x\rangle.\,M \colon a(\iota)\,\overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{abs}}{\Gamma + \Delta \vdash_{\mathsf{w}} [N]a.\,a\langle x\rangle.\,M \colon \overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{app} \qquad \dfrac{\Gamma \vdash N \colon \iota \qquad \Delta,\, x \colon \iota \vdash M \colon \overline{\kappa} \Rightarrow \overline{\mu}}{\Gamma + \Delta \vdash \{N/x\}M \colon \overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{sub}$$

- **Passage case:** For a reduction $[N]b.\,a\langle x\rangle.\,M \longrightarrow a\langle x\rangle.\,[N]b.\,M$ where $a \neq b$ and $x \notin \mathsf{fv}(N)$, we have the derivation below left if and only if we have that below right, with equal weights. Note that $a(\iota)\,b(\nu)\,\overline{\kappa} = b(\nu)\,a(\iota)\,\overline{\kappa}$.

$$\dfrac{\Gamma \vdash N \colon \iota \qquad \dfrac{\dfrac{\Delta,\, x \colon \iota \vdash M \colon b(\nu)\,\overline{\kappa} \Rightarrow \overline{\mu}}{\Delta \vdash a\langle x\rangle.\,M \colon a(\iota)\,b(\nu)\,\overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{abs}}{\Gamma + \Delta \vdash_{\mathsf{w}} [N]b.\,a\langle x\rangle.\,M \colon a(\iota)\,\overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{app} \qquad \dfrac{\dfrac{\Gamma \vdash N \colon \iota \qquad \Delta,\, x \colon \iota \vdash M \colon b(\nu)\,\overline{\kappa} \Rightarrow \overline{\mu}}{\Gamma + \Delta,\, x \colon \iota \vdash [N]b.\,M \colon \overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{abs}}{\Gamma + \Delta \vdash_{\mathsf{w}} a\langle x\rangle.\,[N]b.\,M \colon a(\iota)\,\overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{app}$$

- **Next case:** For a reduction $\star\,;M \longrightarrow M$, we have the derivation for $\Gamma \vdash^{n+2} \star\,;M \colon \overline{\iota} \Rightarrow \overline{\kappa}$ below if and only if we have a derivation for $\Gamma \vdash^{n} M \colon \overline{\iota} \Rightarrow \overline{\kappa}$.

$$\dfrac{\dfrac{}{\vdash \star \colon \overline{\iota} \Rightarrow \overline{\iota}}\,\mathsf{unit} \qquad \Gamma \vdash M \colon \overline{\iota} \Rightarrow \overline{\kappa}}{\Gamma \vdash \star\,;M \colon \overline{\iota} \Rightarrow \overline{\kappa}}\,\mathsf{seq}$$

- **Prefix (pop) case:** For a reduction $a\langle x\rangle.\,N\,;M \longrightarrow a\langle x\rangle.\,(N\,;M)$ where $x \notin \mathsf{fv}(M)$, given the derivation below left we may construct that below right, and vice versa.

$$\dfrac{\dfrac{\Gamma,\, x \colon \iota \vdash N \colon \overline{\kappa} \Rightarrow \overline{\mu}}{\Gamma \vdash a\langle x\rangle.\,N \colon a(\iota)\,\overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{abs} \qquad \Delta \vdash M \colon \overline{\mu} \Rightarrow \overline{\nu}}{\Gamma + \Delta \vdash a\langle x\rangle.\,N\,;M \colon a(\iota)\,\overline{\kappa} \Rightarrow \overline{\nu}}\,\mathsf{seq} \qquad \dfrac{\dfrac{\Gamma,\, x \colon \iota \vdash N \colon \overline{\kappa} \Rightarrow \overline{\mu} \qquad \Delta \vdash M \colon \overline{\mu} \Rightarrow \overline{\nu}}{\Gamma + \Delta,\, x \colon \iota \vdash N\,;M \colon \overline{\kappa} \Rightarrow \overline{\nu}}\,\mathsf{seq}}{\Gamma + \Delta \vdash a\langle x\rangle.\,(N\,;M) \colon a(\iota)\,\overline{\kappa} \Rightarrow \overline{\nu}}\,\mathsf{abs}$$

- **Prefix (push) case:** For a reduction $[P]a.\,N\,;M \longrightarrow [P]a.\,(N\,;M)$, given the first derivation below we may construct the second, and vice versa.

$$\dfrac{\dfrac{\Gamma \vdash P \colon \iota \qquad \Delta \vdash N \colon a(\iota)\,\overline{\kappa} \Rightarrow \overline{\mu}}{\Gamma + \Delta \vdash_{\mathsf{w}} [P]a.\,N \colon \overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{app} \qquad \Lambda \vdash M \colon \overline{\mu} \Rightarrow \overline{\nu}}{\Gamma + \Delta + \Lambda \vdash [P]a.\,N\,;M \colon \overline{\kappa} \Rightarrow \overline{\nu}}\,\mathsf{seq}$$

$$\dfrac{\Gamma \vdash P : \iota \qquad \dfrac{\Delta \vdash N : a(\iota)\,\overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}} \qquad \Lambda \vdash M : \overline{\overline{\mu}} \Rightarrow \overline{\overline{\nu}}}{\Delta + \Lambda \vdash N\,;M : a(\iota)\,\overline{\overline{\kappa}} \Rightarrow \overline{\overline{\nu}}}\,\mathsf{seq}}{\Gamma + \Delta + \Lambda \vdash_{\mathsf{w}} [P]a.\,(N\,;M) : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\nu}}}\,\mathsf{app}$$

- **Associate case:** For a reduction $(P\,;N)\,;M \longrightarrow P\,;(N\,;M)$, given the first derivation below we may construct the second, and vice versa.

$$\dfrac{\dfrac{\Gamma \vdash P : \overline{\iota} \Rightarrow \overline{\overline{\kappa}} \qquad \Delta \vdash N : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}}{\Gamma + \Delta \vdash P\,;N : \overline{\iota} \Rightarrow \overline{\overline{\kappa}}}\,\mathsf{seq} \qquad \Lambda \vdash M : \overline{\overline{\mu}} \Rightarrow \overline{\overline{\nu}}}{\Gamma + \Delta + \Lambda \vdash (P\,;N)\,;M : \overline{\iota} \Rightarrow \overline{\overline{\nu}}}\,\mathsf{seq}$$

$$\dfrac{\Gamma \vdash P : \overline{\iota} \Rightarrow \overline{\overline{\kappa}} \qquad \dfrac{\Delta \vdash N : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}} \qquad \Lambda \vdash M : \overline{\overline{\mu}} \Rightarrow \overline{\overline{\nu}}}{\Delta + \Lambda \vdash N\,;M : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\nu}}}\,\mathsf{seq}}{\Gamma + \Delta + \Lambda \vdash P\,;(N\,;M) : \overline{\iota} \Rightarrow \overline{\overline{\nu}}}\,\mathsf{seq}$$

For reduction in a spine context, $W\{M\} \rightarrow_{\mathsf{w}} W\{N\}$, it is sufficient to observe from the derivation rules that the derivation for M occurs exactly once in $W\{M\}$; then the effect of replacing the derivation for $\Gamma \vdash_{\mathsf{w}} M : \tau$ with $\Gamma \vdash_{\mathsf{w}} N : \tau$, or vice versa, reflects directly on the derivations for $W\{M\}$ and $W\{N\}$. □

Lemma 36 (Spine normal forms terminate)**.** *For a spine-normal term $\mathcal{W}$ there is a dimension d such that for any memory S_A and substitution map σ of dimension d, there is a memory T_A such that $S_A, \sigma\mathcal{W} \Downarrow T_A$.*

Proof. By induction on $\mathcal{W}$ it will be shown that there is a dimension d such that for any memory S_A and substitution map σ of dimension d, the following hold.

(1) There is a memory T_A such that $S_A, \sigma\mathcal{W} \Downarrow T_A$.
(2) For a term $\mathcal{V}$, there is a memory T_A such that $\varepsilon, \sigma\mathcal{V} \Downarrow T_A$.

Observe that case 1 implies case 2 by expanding the memory ε to S_A.

- **Unit case:** We prove statement 2. Let $d = 0$. We have $\sigma(\star) = \star$ and $\varepsilon, \star \Downarrow \varepsilon$.
- **Variable case:** We prove statement 2. Let $d = 0$. Given σ, since σx is a term of dimension d, we have $\varepsilon, \sigma x \Downarrow T_A$ for some memory T_A of dimension d.
- **Sequencing case:** We prove statement 2. Given $x\,;\mathcal{W}$, by induction we get a dimension d such that statement 1 holds for $\mathcal{W}$. Let σ be a substitution map of dimension d. As in the previous case, we have $\varepsilon, \sigma x \Downarrow S_A$ where S_A is a memory of dimension d. Then by the inductive hypothesis, we have $S_A, \sigma\mathcal{W} \Downarrow T_A$, and by the evaluation rule for sequencing, $\varepsilon, \sigma(x\,;\mathcal{W}) \Downarrow T_A$.

$$\dfrac{\varepsilon, \sigma x \Downarrow S_A \qquad S_A, \sigma\mathcal{W} \Downarrow T_A}{\varepsilon, \sigma(x\,;\mathcal{W}) \Downarrow T_A}$$

- **Application case:** We prove statement 2. Given $[M]a.\,\mathcal{V}$, by induction we get a dimension d such that $\varepsilon, \sigma\mathcal{V} \Downarrow T_A$ given any σ of dimension d. Then by expansion and the evaluation rule for application, statement 2 follows for $[M]a.\,\mathcal{V}$ with dimension d and return memory $\sigma M_a \cdot T_A$ (abusing notation to prepend σM to the stack a in T_A).

$$\dfrac{\sigma M_a, \sigma\mathcal{V} \Downarrow \sigma M_a \cdot T_A}{\varepsilon, \sigma([M]a.\,\mathcal{V}) \Downarrow \sigma M_a \cdot T_A}$$

- **Abstraction case:** Given $a\langle x\rangle.\,\mathcal{W}$, by induction there is a dimension d such that statement 1 holds for $\mathcal{W}$. We prove statement 1 for dimension $d+1$. A memory of dimension $d+1$ is of the form $S_A \cdot M_a$ where both M and S_A are of dimension d, in the latter case since every stack is of length at least d (and holding terms of dimenstion at least d). A substitution map σ of dimension $d+1$ is also of dimension d. Then so is $\sigma\{M/x\}$ (which assigns

M to x and behaves as σ everywhere else). The inductive hypothesis then gives $S_A,\, \sigma\{M/x\}\mathcal{W} \Downarrow T_A$. By the evaluation rule for abstraction, from this we get $S_A \cdot M_a,\, \sigma(a\langle x\rangle.\, \mathcal{W}) \Downarrow T_A$ for dimension $d+1$.

$$\frac{S_A,\, \sigma\{M/x\}\mathcal{W} \Downarrow T_A}{S_A \cdot M_a,\, \sigma(a\langle x\rangle.\, \mathcal{W}) \Downarrow T_A}$$

□

Lemma 37 (Evaluation is invariant under spine reduction)**.** *If $M \twoheadrightarrow_{\mathsf{w}} N$ then $S_A,\, \sigma M \Downarrow T_A$ if and only if $S_A,\, \sigma N \Downarrow T_A$ for any memories S_A and T_A and substitution map σ.*

Proof. By induction on the spine context for the reduction step. The first six cases cover reduction at top level, and the last four reduction in context.

- **Beta case:** Let $[N]a.\, a\langle x\rangle.\, M \longrightarrow \{N/x\}M$. Noting that $\sigma(\{N/x\}M) = (\sigma\{\sigma N/x\})M$, the left hand term evaluates as below if and only if the right hand term evaluates as $S_A,\, \sigma(\{N/x\}M) \Downarrow T_A$.

$$\frac{\dfrac{S_A,\, \sigma(\{N/x\}M) \Downarrow T_A}{S_A \cdot \sigma N_a,\, \sigma(a\langle x\rangle.\, M) \Downarrow T_A}}{S_A,\, \sigma([N]a.\, a\langle x\rangle.\, M) \Downarrow T_A}$$

- **Passage case:** Let $[N]b.\, a\langle x\rangle.\, M \longrightarrow a\langle x\rangle.\, [N]b.\, M$ where x is not free in N. Assuming by alpha-conversion that σ does not bind in P, we have the first derivation if and only if we have the second.

$$\frac{\dfrac{S_A \cdot \sigma N_b,\, \sigma(\{P/x\}M) \Downarrow T_A}{S_A \cdot P_a \cdot \sigma N_b,\, \sigma(a\langle x\rangle.\, M) \Downarrow T_A}}{S_A \cdot P_a,\, \sigma([N]b.\, a\langle x\rangle.\, M) \Downarrow T_A} \qquad \frac{\dfrac{S_A \cdot \sigma N_b,\, \sigma(\{P/x\}M) \Downarrow T_A}{S_A,\, \sigma([N]b.\, \{P/x\}M) \Downarrow T_A}}{S_A \cdot P_a,\, \sigma(a\langle x\rangle.\, [N]b.\, M) \Downarrow T_A}$$

- **Next case:** Let $\star\,;M \longrightarrow M$. Since $\sigma(\star) = \star$, we have $S_A,\, \sigma M \Downarrow T_A$ if and only if we have the folowing.

$$\frac{S_A,\, \star \Downarrow S_A \qquad S_A,\, \sigma M \Downarrow T_A}{S_A,\, \sigma(\star\,; M) \Downarrow T_A}$$

- **Prefix (pop) case:** Let $a\langle x\rangle.\, N\,;M \longrightarrow a\langle x\rangle.\, (N\,;M)$ where x is not free in M. Assuming by alpha-conversion that σ does not bind in P, we have the first derivation if and only if we have the second.

$$\frac{\dfrac{R_A,\, \sigma(\{P/x\}N) \Downarrow S_A}{R_A \cdot P_a,\, \sigma(a\langle x\rangle.\, N) \Downarrow S_A} \qquad S_A,\, \sigma M \Downarrow T_A}{R_A \cdot P_a,\, \sigma(a\langle x\rangle.\, N\,;M) \Downarrow T_A} \qquad \frac{R_A,\, \sigma(\{P/x\}N) \Downarrow S_A \qquad S_A,\, \sigma M \Downarrow T_A}{\dfrac{R_A,\, \sigma(\{P/x\}N\,;M) \Downarrow T_A}{R_A \cdot P_a,\, \sigma(a\langle x\rangle.\, (N\,;M)) \Downarrow T_A}}$$

- **Prefix (push) case:** For a reduction $[P].\, N\,;M \longrightarrow [P].\, (N\,;M)$, we have the first derivation if and only if we have the second.

$$\frac{\dfrac{R_A \cdot \sigma P_a,\, \sigma N \Downarrow S_A}{R_A,\, \sigma([P].\, N) \Downarrow S_A} \qquad S_A,\, \sigma M \Downarrow T_A}{R_A,\, \sigma([P].\, N\,;M) \Downarrow T_A} \qquad \frac{R_A \cdot \sigma P_a,\, \sigma N \Downarrow S_A \qquad S_A,\, \sigma M \Downarrow T_A}{\dfrac{R_A \cdot \sigma P_a,\, \sigma(N\,;M) \Downarrow T_A}{R_A,\, \sigma([P].\, (N\,;M)) \Downarrow T_A}}$$

- **Associativity case:** For a reduction $(P\,;N)\,;M \longrightarrow P\,;(N\,;M)$, we have the first derivation if and only if we have the second.

$$\frac{\dfrac{\dfrac{R_A,\, \sigma P \Downarrow S_A \qquad S_A,\, \sigma N \Downarrow T_A}{R_A,\, \sigma(P\,;N) \Downarrow T_A} \qquad T_A,\, \sigma M \Downarrow U_A}{R_A,\, \sigma((P\,;N)\,;M) \Downarrow U_A}}{} \qquad \frac{R_A,\, \sigma P \Downarrow S_A \qquad \dfrac{S_A,\, \sigma N \Downarrow T_A \qquad T_A,\, \sigma M \Downarrow U_A}{S_A,\, \sigma(N\,;M) \Downarrow U_A}}{R_A,\, \sigma(P\,;(N\,;M)) \Downarrow U_A}$$

- **Application case:** Let $[P]a.\,M \longrightarrow [P]a.\,N$ where $M \longrightarrow N$. Given S_A and σ, by induction we have $S_A \cdot \sigma P_a$, $\sigma M \Downarrow T_A$ if and only if $S_A \cdot \sigma P_a$, $\sigma N \Downarrow T_A$, and hence S_A, $\sigma([P]a.\,M) \Downarrow T_A$ if and only if S_A, $\sigma([P]a.\,N) \Downarrow T_A$.
- **Abstraction case:** Let $a\langle x\rangle.\,M \longrightarrow a\langle x\rangle.\,N$ where $M \longrightarrow N$. Given $S_A \cdot \sigma P_a$ and σ, assuming σ does not bind in P, by induction we have S_A, $\sigma(\{P/x\}M) \Downarrow T_A$ if and only if S_A, $\sigma(\{P/x\}N) \Downarrow T_A$, and hence $S_A \cdot \sigma P_a$, $\sigma(a\langle x\rangle.\,M) \Downarrow T_A$ if and only if $S_A \cdot \sigma P_a$, $\sigma(a\langle x\rangle.\,N) \Downarrow T_A$.
- **Prefix case:** Let $M\,;P \longrightarrow N\,;P$ where $M \longrightarrow N$. Given S, by induction we have R_A, $\sigma M \Downarrow S_A$ if and only if R_A, $\sigma N \Downarrow S_A$, so given S_A, $\sigma P \Downarrow T_A$ we have R_A, $\sigma(M\,;P) \Downarrow T_A$ if and only if S_A, $\sigma(N\,;P) \Downarrow T_A$.
- **Postfix case:** Let $P\,;M \longrightarrow P\,;N$ where $M \longrightarrow N$. By induction we have S_A, $\sigma M \Downarrow T_A$ if and only if S_A, $\sigma N \Downarrow T_A$, so given R_A, $\sigma P \Downarrow S_A$ we have R_A, $\sigma(P\,;M) \Downarrow T_A$ if and only if R_A, $\sigma(P\,;N) \Downarrow T_A$. □

Appendix E. Proofs for section 5: Strong Normalization

Lemma 38 (Substitution, strong)**.** *If there are derivations* $\Gamma \vdash_{\mathsf{s}}^{n} N : \nu$ *and* $x : \nu,\ \Delta \vdash_{\mathsf{s}}^{m} M : \tau$ *then there is a derivation* $\Gamma + \Delta \vdash_{\mathsf{s}}^{k} \{N/x\}M : \tau$ *where* $k \leq n + m$*, and vice versa where* $k = n + m$*.*

Proof. The proof is similar to the substitution lemma for the weak type system, Lemma 9. We cover the new cases: *strong application* and *weakening*.

- **Strong application case:** For a term $M = [Q]a.\,P$ we have the following derivations, both of the same weight, where $\nu = \nu_1 + \nu_2 + \nu_3$, $\Gamma = \Gamma_1 + \Gamma_2 + \Gamma_3$ and $\Delta = \Delta_1 + \Delta_2 + \Delta_3$. In the second derivation, the premises of the top rules are stacked for space and readability. The argument is as follows. Given the first derivation, where $\Gamma \vdash N : \nu$ is introduced by a collection typing rule col, we may partition its premisses along the decomposition $\nu = \nu_1 + \nu_2 + \nu_3$ to get $\Gamma_i \vdash N : \nu_i$ for $i = 1, 2, 3$. Then we may construct the second derivation. In the reverse direction, given the second derivation, we may construct the first.

$$\dfrac{\Gamma \vdash N : \nu \qquad \dfrac{x : \nu_1,\ \Delta_1 \vdash Q : \iota \quad x : \nu_2,\ \Delta_2 \vdash P : a(\iota)\,\overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}} \quad x : \nu_3,\ \Delta_3 \vdash Q : \tau}{x : \nu,\ \Delta \vdash_{\mathsf{s}} [Q]a.\,P : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}}\,\mathsf{APP}}{\Gamma + \Delta \vdash \{N/x\}[Q]a.\,P : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}}\,\mathsf{sub}$$

$$\dfrac{\dfrac{\Gamma_1 \vdash N : \nu_1 \quad x : \nu_1,\ \Delta_1 \vdash Q : \iota}{\Gamma_1 + \Delta_1 \vdash \{N/x\}Q : \iota}\,\mathsf{sub} \qquad \dfrac{\Gamma_2 \vdash N : \nu_2 \quad x : \nu_2,\ \Delta_2 \vdash P : a(\iota)\,\overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}}{\Gamma_2 + \Delta_2 \vdash \{N/x\}P : a(\iota)\,\overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}}\,\mathsf{sub} \qquad \dfrac{\Gamma_3 \vdash N : \nu_3 \quad x : \nu_3,\ \Delta_3 \vdash Q : \tau}{\Gamma_3 + \Delta_3 \vdash \{N/x\}Q : \tau}\,\mathsf{sub}}{\Gamma + \Delta \vdash_{\mathsf{s}} [\{N/x\}Q].\,\{N/x\}P : \overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{APP}$$

- **Weakening case:** From left to right, assume the derivation below left with sub-derivations of weights i and k. The derivation for N, ending in a collection typing rule, may be partitioned along $\nu + \mu$ to give $\Gamma_1 \vdash^{n} N : \nu$ and $\Gamma_2 \vdash^{m} N : \mu$ where $n + m = i$. Then we may construct the derivation below right, which by induction has a weight $j \leq n + k$.

$$\dfrac{\Gamma \vdash^{i} N : \nu + \mu \qquad \dfrac{x : \nu,\ \Delta \vdash^{k} M : \tau}{x : \nu + \mu,\ \Delta + \Lambda \vdash_{\mathsf{s}} M : \tau}\,\mathsf{WEAK}}{\Gamma + \Delta + \Lambda \vdash \{N/x\}M : \tau}\,\mathsf{sub} \qquad \dfrac{\dfrac{\Gamma_1 \vdash^{n} N : \nu \quad x : \nu,\ \Delta \vdash^{k} M : \tau}{\Gamma_1 + \Delta + \Lambda \vdash \{N/x\}M : \tau}\,\mathsf{sub}}{\Gamma_1 + \Gamma_2 + \Delta + \Lambda \vdash \{N/x\}M : \tau}\,\mathsf{WEAK}$$

From right to left, given the derivation below left we may construct that below right of the same weight $n + m$, where by alpha-conversion we may

assume x does not occur in the context Λ.

$$\dfrac{\dfrac{\Gamma \vdash^n N : \nu \qquad x : \nu,\, \Delta \vdash^m M : \tau}{\Gamma + \Delta \vdash \{N/x\}M : \tau}\,\textsf{sub}}{\Gamma + \Delta + \Lambda \vdash \{N/x\}M : \tau}\,\textsf{WEAK} \qquad \dfrac{\Gamma \vdash^n N : \nu \qquad \dfrac{x : \nu,\, \Delta \vdash^m M : \tau}{x : \nu,\, \Delta + \Lambda \vdash M : \tau}\,\textsf{WEAK}}{\Gamma + \Delta + \Lambda \vdash \{N/x\}M : \tau}\,\textsf{sub}$$ □

Lemma 39 (Subject reduction, beta)**.** *If $M \longrightarrow N$ by a beta-step and $\Gamma \vdash_{\mathsf{s}}^m M : \tau$ then $\Gamma \vdash_{\mathsf{s}}^n N : \tau$ where $m > n$.*

Proof. By induction on the reduction context. We give the case for a beta-redex and that for the *argument* context $[C]a.\, M$ of the strong application rule. For the other cases, the derivation for $C\{M\}$ contains exactly one subderivation for M, so that a change in weight is reflected directly.

- **Beta case:** For a reduction $[N]a.\, a\langle x\rangle.\, M \longrightarrow \{N/x\}M$, given the first derivation below we may construct the second. Where the first derivation has weight $n + m + k + 2$, by Lemma 21 the second has weight $i \leq n + m$.

$$\dfrac{\Gamma \vdash^n N : \iota \qquad \dfrac{\dfrac{\Delta,\, x : \iota \vdash^m M : \overline{\kappa} \Rightarrow \overline{\mu}}{\Delta \vdash a\langle x\rangle.\, M : a(\iota)\, \overline{\kappa} \Rightarrow \overline{\mu}}\,\textsf{abs} \qquad \Lambda \vdash^k N : \tau}{\Gamma + \Delta + \Lambda \vdash_{\mathsf{s}} [N]a.\, a\langle x\rangle.\, M : \overline{\kappa} \Rightarrow \overline{\mu}}\,\textsf{APP}$$

$$\dfrac{\Gamma \vdash^n N : \iota \qquad \Delta,\, x : \iota \vdash^m M : \overline{\kappa} \Rightarrow \overline{\mu}}{\dfrac{\Gamma + \Delta \vdash \{N/x\}M : \overline{\kappa} \Rightarrow \overline{\mu}}{\Gamma + \Delta + \Lambda \vdash_{\mathsf{s}} \{N/x\}M : \overline{\kappa} \Rightarrow \overline{\mu}}\,\textsf{WEAK}}\,\textsf{sub}$$

- **Argument case:** For a reduction $[M]a.\, P \longrightarrow [N]a.\, P$ given $M \longrightarrow N$, consider the strong application inference below, where $\Gamma = \Gamma_1 + \ldots + \Gamma_n$ and $\iota = [\tau_1 \ldots \tau_n]$ Observe that there is at least one sub-derivation for M, given by $\Lambda \vdash M : \tau$, in the derivation for $[M]a.\, P$. Replacing every sub-derivation for M with one for N of strictly smaller weight then gives a strictly smaller weight for the derivation for $[N]a.\, P$.

$$\dfrac{\dfrac{(\Gamma_i \vdash M : \tau_i)_{1 \leq i \leq n}}{\Gamma \vdash M : \iota}\,\textsf{col} \qquad \Delta \vdash P : a(\iota)\, \overline{\kappa} \Rightarrow \overline{\mu} \qquad \Lambda \vdash M : \tau}{\Gamma + \Delta \vdash_{\mathsf{s}} [M]a.\, P : \overline{\kappa} \Rightarrow \overline{\mu}}\,\textsf{APP}$$ □

Lemma 40 (Subject reduction and expansion, non-beta)**.** *If $M \longrightarrow N$ by a non-beta-step then $\Gamma \vdash_{\mathsf{s}}^m M : \tau$ if and only if $\Gamma \vdash_{\mathsf{s}}^n N : \tau$ where $m \geq n$.*

Proof. The proof is similar to Lemma 15. The one new case for top-level reduction is the following.

- **Strong application/sequence case:** For a reduction $[P]a.\, N\,;M \longrightarrow [P]a.\, (N\,;M)$, given the first derivation below we may construct the second, of equal weight.

$$\dfrac{\dfrac{\Gamma \vdash P : \iota \qquad \Delta \vdash N : a(\iota)\, \overline{\kappa} \Rightarrow \overline{\mu} \qquad \Lambda \vdash P : \tau}{\Gamma + \Delta + \Lambda \vdash_{\mathsf{s}} [P]a.\, N : \overline{\kappa} \Rightarrow \overline{\mu}}\,\textsf{APP} \qquad \Sigma \vdash M : \overline{\mu} \Rightarrow \overline{\nu}}{\Gamma + \Delta + \Lambda + \Sigma \vdash [P]a.\, N\,;M : \overline{\kappa} \Rightarrow \overline{\nu}}\,\textsf{seq}$$

$$\dfrac{\Gamma \vdash P : \iota \qquad \dfrac{\Delta \vdash N : a(\iota)\, \overline{\kappa} \Rightarrow \overline{\mu} \qquad \Sigma \vdash M : \overline{\mu} \Rightarrow \overline{\nu}}{\Delta + \Sigma \vdash N\,;M : a(\iota)\, \overline{\kappa} \Rightarrow \overline{\nu}}\,\textsf{seq} \qquad \Lambda \vdash P : \tau}{\Gamma + \Delta + \Lambda + \Sigma \vdash_{\mathsf{s}} [P]a.\, (N\,;M) : \overline{\kappa} \Rightarrow \overline{\nu}}\,\textsf{APP}$$

For reduction in context, $C\{M\} \longrightarrow C\{N\}$, the new case is the argument position, where there may be any (non-zero) number of sub-derivations for M in $C\{M\}$. Replacing a sub-derivation with one of equal or lower weight then results in an overall weight that is equal or lower. □

Appendix F. Proofs for Section 6: Perpetual Evaluation

Proposition 41 (Normal forms)**.** *The normal forms of* $\longrightarrow$ *are the terms* $\mathcal{M}$ *given by the following grammars.*

$$\begin{array}{rcl} \mathcal{M} & ::= & a\langle x\rangle.\,\mathcal{M} \mid \mathcal{N} \\ \mathcal{N} & ::= & [\mathcal{M}]a.\,\mathcal{N} \mid x\,;\mathcal{M} \mid x \mid \star \end{array}$$

Proof. The proof is analogous to that of Proposition 13. Terms $\mathcal{M}$ are the normal forms, and $\mathcal{N}$ those that are not abstractions. For the latter, variables x and the unit $\star$ are normal, and a composition $M\,;N$ is normal if M is a variable and N is normal, and thus of the form $x\,;\mathcal{M}$. An application $[N]a.\,M$ is normal if N and M are and M is not an abstraction, thus of the form $[\mathcal{M}].\,\mathcal{N}$. Finally, an abstraction $a\langle x\rangle.\,M$ is normal if M is, thus of the form $a\langle x\rangle.\,\mathcal{M}$. □

Lemma 42 (Strong normalization implies perpetual evaluation)**.** *If* M *is strongly normalizing then* $M \blacktriangleright \mathcal{M}$ *and* $M \twoheadrightarrow \mathcal{M}$ *for some* $\mathcal{M}$*.*

Proof. The proof is by induction on the following pair of measures: first, the sum length of reduction paths from M, and second, the size of M. Let M be of the form $[S_A].\,M'$, with M' not an application and proceed by case analysis on the structure of M'.

- **Unit case:** if M is of the form $[S_A].\,\star$ then we have the case *Normal (unit)* of the definition of $\blacktriangleright$ as below. The premisses are given by the inductive hypothesis: each term N_i has equal or fewer reductions than M, and is smaller. The reduction $M \twoheadrightarrow \mathcal{M}$ follows from the reductions $N_i \twoheadrightarrow \mathcal{M}_i$.

$$\frac{(N_i \blacktriangleright \mathcal{M}_i)_{i\leq n}}{[N_1]a_1 \ldots [N_n]a_n.\,\star \blacktriangleright [\mathcal{M}_1]a_1 \ldots [\mathcal{M}_n]a_n.\,\star}$$

- **Variable case:** if M is of the form $[S_A].\,x$ then we have the case *Normal (variable)* as below. It resolves as the previous case.

$$\frac{(N_i \blacktriangleright \mathcal{M}_i)_{i\leq n}}{[N_1]a_1 \ldots [N_n]a_n.\,x \blacktriangleright [\mathcal{M}_1]a_1 \ldots [\mathcal{M}_n]a_n.\,x}$$

- **Abstraction case:** if M' an abstraction $a\langle x\rangle.\,N$ there are three sub-cases. If $[S_A]$ is empty, we have the case *Normal (abstraction)* as below.

$$\frac{N \blacktriangleright \mathcal{M}}{a\langle x\rangle.\,N \blacktriangleright a\langle x\rangle.\,\mathcal{M}}$$

  If $[S_A]$ is of the form $[T_A].\,[P]a$, ending in a push on location a, we have the case *Beta* as below, with the reduction $[T_A].\,[P]a.\,a\langle x\rangle.\,N \longrightarrow [T_A].\,\{P/x\}N \longrightarrow \mathcal{M}$.

$$\frac{[T_A].\,\{P/x\}N \blacktriangleright \mathcal{M}}{[T_A].\,[P]a.\,a\langle x\rangle.\,N \blacktriangleright \mathcal{M}}$$

  If $[S_A]$ is of the form $[T_A].\,[P]b$, ending in a push on location $b \neq a$, we have the case *Passage* as below, with the reduction $[T_A].\,[P]b.\,a\langle x\rangle.\,N \longrightarrow [T_A].\,a\langle x\rangle.\,[P]b.\,N \longrightarrow \mathcal{M}$.

$$\frac{[T_A].\,a\langle x\rangle.\,[P]b.\,N \blacktriangleright \mathcal{M}}{[T_A].\,[P]b.\,a\langle x\rangle.\,N \blacktriangleright \mathcal{M}}$$

- **Unit/sequence case:** If M is of the form $[S_A].\,(\star\,;P)$ we have the case *Next* below, with the reduction $[S_A].\,(\star\,;P) \longrightarrow [S_A].\,P \twoheadrightarrow \mathcal{M}$.

$$\frac{[S_A].\,P \blacktriangleright \mathcal{M}}{[S_A].\,(\star\,;P) \blacktriangleright \mathcal{M}}$$

- **Variable/sequence case:** If M is of the form $[S_A].\,(x\,;P)$ we have the case *Normal (sequence)* as below. The inductive hypothesis gives the premisses, which each have a smaller measure. The reduction from M follows by that for each N_i and P.

$$\frac{(N_i \blacktriangleright \mathcal{M}_i)_{i\leq n} \qquad P \blacktriangleright \mathcal{M}}{[N_1]a_1 \ldots [N_n]a_n.\,(x\,;P) \blacktriangleright [\mathcal{M}_1]a_1 \ldots [\mathcal{M}_n]a_n.\,(x\,;\mathcal{M})}$$

- **Abstraction/sequence case:** If M is of the form $[S_A].\,(a\langle x\rangle.\,N\,;P)$ we have the case *Prefix (pop)* as below, with the reduction $[S_A].\,(a\langle x\rangle.\,N\,;P) \longrightarrow [S_A].\,a\langle x\rangle.\,(N\,;P) \twoheadrightarrow \mathcal{M}$.

$$\frac{[S_A].\,a\langle x\rangle.\,(N\,;P) \blacktriangleright \mathcal{M}}{[S_A].\,(a\langle x\rangle.\,N\,;P) \blacktriangleright \mathcal{M}}$$

- **Application/sequence case:** If M is of the form $[S_A].\,([P]a.\,N\,;Q)$ we have the case *Prefix (push)* as below, with the reduction $[S_A].\,([P]a.\,N\,;Q) \longrightarrow [S_A].\,[P]a.\,(N\,;Q) \twoheadrightarrow \mathcal{M}$.

$$\frac{[S_A].\,[P]a.\,(N\,;Q) \blacktriangleright \mathcal{M}}{[S_A].\,([P]a.\,N\,;Q) \blacktriangleright \mathcal{M}}$$

- **Sequence/sequence case:** If M is of the form $[S_A].\,((Q\,;P)\,;N)$ we have the case *Associate* as below, with the reduction $[S_A].\,((Q\,;P)\,;N) \longrightarrow [S_A].\,(Q\,;(P\,;N)) \twoheadrightarrow \mathcal{M}$.

$$\frac{[S_A].\,(Q\,;(P\,;N)) \blacktriangleright \mathcal{M}}{[S_A].\,((Q\,;P)\,;N) \blacktriangleright \mathcal{M}}$$

□

Lemma 43 (Perpetual evaluation implies strong typeability)**.** *If* $M \blacktriangleright \mathcal{M}$ *then* $\Gamma \vDash_{\mathsf{s}} M : \tau$ *and* $\Gamma \vDash_{\mathsf{s}} \mathcal{M} : \tau$ *for some* Γ *and* τ.

Proof. For the notation $[S_A].\,M$ we introduce an associated typing rule APPS (*applications*), and a strong typing rule for memories M1.

$$\frac{\Gamma \vDash_{\mathsf{s}} S_A : \overline{\iota} \qquad \Delta \vDash_{\mathsf{s}} M : \overline{\iota}\,\overline{\kappa} \Rightarrow \overline{\mu}}{\Gamma + \Delta \vDash_{\mathsf{s}} [S_A].\,M : \overline{\kappa} \Rightarrow \overline{\mu}}\,\mathsf{APPS} \qquad \frac{\Gamma \vDash_{\mathsf{s}} S_A : \overline{\iota} \qquad \Delta \vDash_{\mathsf{s}} N : \kappa \qquad \Lambda \vDash_{\mathsf{s}} N : \tau}{\Gamma + \Delta + \Lambda \vDash_{\mathsf{s}} S_A \cdot N_a : \overline{\iota}\,a(\kappa)}\,\mathsf{M1}$$

By induction, these rule are admissible: we have the first derivation below if and only if we have the second.

$$\dfrac{\dfrac{\Gamma \vDash_{\mathsf{s}} S_A : \overline{\iota} \quad \Delta \vDash_{\mathsf{s}} N : \kappa \quad \Lambda \vDash_{\mathsf{s}} N : \tau}{\Gamma + \Delta + \Lambda \vDash_{\mathsf{s}} S_A \cdot N_a : \overline{\iota}\,a(\kappa)}\,\mathsf{M1} \qquad \Sigma \vDash_{\mathsf{s}} M : a(\kappa)\,\overline{\iota}\,\overline{\mu} \Rightarrow \overline{\nu}}{\Gamma + \Delta + \Lambda + \Sigma \vDash_{\mathsf{s}} [S_A \cdot N_a].\,M}\,\mathsf{APPS}$$

$$\dfrac{\Gamma \vDash_{\mathsf{s}} S_A : \overline{\iota} \qquad \dfrac{\Delta \vDash_{\mathsf{s}} N : \kappa \quad \Lambda \vDash_{\mathsf{s}} N : \tau \quad \Sigma \vDash_{\mathsf{s}} M : a(\kappa)\,\overline{\iota}\,\overline{\mu} \Rightarrow \overline{\nu}}{\Delta + \Lambda + \Sigma \vDash_{\mathsf{s}} [N]a.\,M : \overline{\iota}\,\overline{\mu} \Rightarrow \overline{\nu}}\,\mathsf{APP}}{\Gamma + \Delta + \Lambda + \Sigma \vDash_{\mathsf{s}} [S_A].\,[N]a.\,M : \overline{\mu} \Rightarrow \overline{\nu}}\,\mathsf{APPS}$$

Again by induction, a memory $S_A = \varepsilon \cdot (N_1)_{a_1} \cdots (N_n)_{a_n}$ may be typed $[]^f$ where $f = |S_A|$, if there is a type for each N_i. This gives the following admissible rules.

$$\frac{(\Gamma_i \vdash N_i : \tau_i)_{i\leq n}}{\Gamma_1 + \ldots + \Gamma_n \vDash_{\mathsf{s}} S_A : []^f} \qquad \frac{(\Gamma_i \vdash N_i : \tau_i)_{i\leq n} \qquad \Delta \vdash M : []^f\,\overline{\iota} \Rightarrow \overline{\kappa}}{\Gamma_1 + \ldots + \Gamma_n + \Delta \vDash_{\mathsf{s}} [S_A].\,M : \overline{\iota} \Rightarrow \overline{\kappa}}$$

The proof is then by induction on the relation $\blacktriangleright$.

- **Beta case:**

$$\frac{[S_A].\,\{N/x\}P \blacktriangleright \mathcal{M} \qquad N \blacktriangleright \mathcal{M}'}{[S_A].\,[N]a.\,a\langle x\rangle.\,P \blacktriangleright \mathcal{M}}$$

The inductive hypothesis gives the following three derivations: that below (using the strong substitution lemma, Lemma 21), $\Sigma \vdash N : \tau$, and $\Gamma+\Delta+\Lambda \vdash \mathcal{M} : \overline{\overline{\mu}} \Rightarrow \overline{\overline{\nu}}$.

$$\dfrac{\Gamma \vdash S_A : \overline{\overline{\iota}} \qquad \dfrac{\Delta \vdash N : \kappa \quad \Lambda, x : \kappa \vdash M : \overline{\overline{\iota}}\,\overline{\overline{\mu}} \Rightarrow \overline{\overline{\nu}}}{\Delta+\Lambda \vdash \{N/x\}P : \overline{\overline{\iota}}\,\overline{\overline{\mu}} \Rightarrow \overline{\overline{\nu}}}\,\mathsf{sub}}{\Gamma+\Delta+\Lambda \vdash [S_A].\{N/x\}P : \overline{\overline{\mu}} \Rightarrow \overline{\overline{\nu}}}\,\mathsf{APPS}$$

From these we may construct the following derivations, the first for M, the second for $\mathcal{M}$.

$$\dfrac{\Gamma \vdash S_A : \overline{\overline{\iota}} \qquad \dfrac{\Delta \vdash N : \kappa \quad \dfrac{\Lambda, x : \kappa \vdash M : \overline{\overline{\iota}}\,\overline{\overline{\mu}} \Rightarrow \overline{\overline{\nu}}}{\Lambda \vdash a\langle x\rangle.P : a(\kappa)\,\overline{\overline{\iota}}\,\overline{\overline{\mu}} \Rightarrow \overline{\overline{\nu}}}\,\mathsf{abs} \qquad \Sigma \vdash N : \tau}{\Delta+\Lambda+\Sigma \vdash [N]a.a\langle x\rangle.P : \overline{\overline{\iota}}\,\overline{\overline{\mu}} \Rightarrow \overline{\overline{\nu}}}\,\mathsf{APP}}{\Gamma+\Delta+\Lambda+\Sigma \vdash [S_A].[N]a.a\langle x\rangle.P}\,\mathsf{APPS}$$

$$\dfrac{\Gamma+\Delta+\Lambda \vdash \mathcal{M} : \overline{\overline{\mu}} \Rightarrow \overline{\overline{\nu}}}{\Gamma+\Delta+\Lambda+\Sigma \vdash \mathcal{M} : \overline{\overline{\mu}} \Rightarrow \overline{\overline{\nu}}}\,\mathsf{WEAK}$$

- **Passage, Next, Prefix (pop), Prefix (push), and Associate cases:** All five cases are of the following form, where $M \longrightarrow N$ by a non-beta reduction.

$$\dfrac{N \blacktriangleright \mathcal{M}}{M \blacktriangleright \mathcal{M}}$$

The inductive hypothesis gives derivations $\Gamma \vDash_{\mathsf{s}} N : \tau$ and $\Gamma \vDash_{\mathsf{s}} \mathcal{M} : \tau$. The required derivation $\Gamma \vDash_{\mathsf{s}} M : \tau$ follows by Lemma 23 (non-beta reduction preserves and reflects typing).

- **Normal (abstraction) case:**

$$\dfrac{M \blacktriangleright \mathcal{M}}{a\langle x\rangle.M \blacktriangleright a\langle x\rangle.\mathcal{M}}$$

The inductive hypothesis gives $\Gamma, x : \iota \vdash M : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}$ and $\Gamma, x : \iota \vdash \mathcal{M} : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}$ (where $\iota = []$ if x does not occur). The abstraction typing rule abs gives $\Gamma \vdash a\langle x\rangle.M : a(\iota)\,\overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}$ and $\Gamma \vdash a\langle x\rangle.\mathcal{M} : a(\iota)\,\overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}$.

- **Normal (unit) case:**

$$\dfrac{(N_i \blacktriangleright \mathcal{M}_i)_{i\leq n}}{[N_1]a_1 \ldots [N_n]a_n.\star \blacktriangleright [\mathcal{M}_1]a_1 \ldots [\mathcal{M}_n]a_n.\star}$$

The inductive hypothesis gives $\Gamma_i \vdash N_i : \tau_i$ and $\Gamma_i \vdash \mathcal{M}_i : \tau_i$ for each $i \leq n$. By the admissible rules introduced above, we have the following derivation for M, and similarly for $\mathcal{M}$, where f is the dimension of the memory $\varepsilon \cdot (N_1)_{a_1} \cdot \cdots \cdot (N_n)_{a_n}$.

$$\dfrac{(\Gamma_i \vdash N_i : \tau_i)_{i\leq n} \qquad \vdash \star : []^f \Rightarrow []^f}{\Gamma_1 + \ldots + \Gamma_n \vdash [N_1]a_1 \ldots [N_n]a_n.\star : \varepsilon \Rightarrow []^f}$$

- **Normal (variable) case:**

$$\dfrac{(N_i \blacktriangleright \mathcal{M}_i)_{i\leq n}}{[N_1]a_1 \ldots [N_n]a_n.x \blacktriangleright [\mathcal{M}_1]a_1 \ldots [\mathcal{M}_n]a_n.x}$$

The case is as the previous one, with the following derivation for M.

$$\dfrac{(\Gamma_i \vdash N_i : \tau_i)_{i\leq n} \qquad x : [\,[]^f \Rightarrow \varepsilon\,] \vdash x : []^f \Rightarrow \varepsilon}{\Gamma_1 + \ldots + \Gamma_n + x : [\,[]^f \Rightarrow \varepsilon\,] \vdash [N_1]a_1 \ldots [N_n]a_n.x : \varepsilon \Rightarrow \varepsilon}$$

- **Normal (sequence) case:**

$$\frac{(N_i \blacktriangleright \mathcal{M}_i)_{i\leq n} \qquad M \blacktriangleright \mathcal{M}}{[N_1]a_1 \dots [N_n]a_n.\,(x\,;M) \blacktriangleright [\mathcal{M}_1]a_1 \dots [\mathcal{M}_n]a_n.\,(x\,;\mathcal{M})}$$

The inductive hypothesis gives $\Delta \vdash M : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}$ and $\Delta \vdash \mathcal{M} : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}$. The case is then as the previous two, with the following derivation.

$$\dfrac{(\Gamma_i \vdash N_i : \tau_i)_{i\leq n} \qquad \dfrac{x : [\,[]^f \Rightarrow \overline{\overline{\kappa}}\,] \vdash x : []^f \Rightarrow \overline{\overline{\kappa}} \quad \Delta \vdash M : \overline{\overline{\kappa}} \Rightarrow \overline{\overline{\mu}}}{x : [\,[]^f \Rightarrow \overline{\overline{\kappa}}\,] + D \vdash x\,;M : []^f \Rightarrow \overline{\overline{\mu}}}\,\mathsf{seq}}{\Gamma_1 + \dots + \Gamma_n + x : [\,[]^f \Rightarrow \overline{\overline{\kappa}}\,] + D \vdash [N_1]a_1 \dots [N_n]a_n.\,(x\,;M) : \varepsilon \Rightarrow \overline{\overline{\mu}}}$$

□

References

[1] Sandra Alves, Delia Kesner, and Miguel Ramos. Quantitative global memory. In Helle Hvid Hansen, Andre Scedrov, and Ruy J. G. B. de Queiroz, editors, *29th International Workshop on Logic, Language, Information, and Computation (WoLLIC)*, volume 13923 of *Lecture Notes in Computer Science*, pages 53–68. Springer, 2023. doi:10.1007/978-3-031-39784-4_4.

[2] Melissa Antonelli, Ugo Dal Lago, and Paolo Pistone. Curry and Howard meet Borel. In *Proceedings of the 37th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS'22)*, pages 45:1–45:13. ACM, 2022. doi:10.1145/3531130.3533361.

[3] Hendrik Pieter Barendregt. *The Lambda Calculus – Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, 1984. doi:10.1016/c2009-0-14341-6.

[4] Chris Barrett, Daniel Castle, and Willem Heijltjes. The Relational Machine Calculus. In Pawel Sobocinski, Ugo Dal Lago, and Javier Esparza, editors, *Proc. 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 9:1–9:15. ACM, 2024. doi:10.1145/3661814.3662091.

[5] Chris Barrett, Willem Heijltjes, and Guy McCusker. The Functional Machine Calculus II: Semantics. In Bartek Klin and Elaine Pimentel, editors, *31st EACSL Annual Conference on Computer Science Logic (CSL 2023)*, volume 252 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 10:1–10:18, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/opus/volltexte/2023/17471, doi:10.4230/LIPIcs.CSL.2023.10.

[6] Jan A. Bergstra and Jan Willem Klop. Strong normalization and perpetual reductions in the lambda calculus. *J. Inf. Process. Cybern.*, 18(7/8):403–417, 1982.

[7] Lasse Blaauwbroek. On the interaction between unrestricted union and intersection types and computational effects. Master's thesis, Eindhoven University of Technology, 2017. URL: https://research.tue.nl/en/studentTheses/d2c9d434-4f17-402e-a757-97df04ac1646.

[8] Viviana Bono and Mariangiola Dezani-Ciancaglini. A tale of intersection types. In Holger Hermanns, Lijun Zhang, Naoki Kobayashi, and Dale Miller, editors, *LICS '20: 35th Annual ACM/IEEE Symposium on Logic in Computer Science, Saarbrücken, Germany, July 8-11, 2020*, pages 7–20. ACM, 2020. doi:10.1145/3373718.3394733.

[9] Gérard Boudol. The lambda-calculus with multiplicities. In *International Conference on Concurrency Theory (CONCUR)*, 1993.

[10] Flavien Breuvart and Ugo Dal Lago. On intersection types and probabilistic lambda calculi. In David Sabel and Peter Thiemann, editors, *Proceedings of the 20th International Symposium on Principles and Practice of Declarative Programming, PPDP 2018, Frankfurt am Main, Germany, September 03-05, 2018*, pages 8:1–8:13. ACM, 2018. doi:10.1145/3236950.3236968.

[11] Antonio Bucciarelli, Delia Kesner, and Daniel Ventura. Non-idempotent intersection types for the lambda-calculus. *Logic Journal of the IGPL*, 25(4):431–464, 2017. doi:10.1093/JIGPAL/JZX018.

[12] M. Coppo and M. Dezani-Ciancaglini. An extension of the basic functionality theory for the λ-calculus. *Notre Dame Journal of Formal Logic*, 21(4):685–693, 1980.

[13] Rowan Davies and Frank Pfenning. Intersection types and computational effects. In Martin Odersky and Philip Wadler, editors, *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00), Montreal, Canada, September 18-21, 2000*, pages 198–208. ACM, 2000. doi:10.1145/351240.351259.

[14] Ugo de'Liguoro and Riccardo Treglia. Intersection types for a λ-calculus with global store. In Niccolò Veltri, Nick Benton, and Silvia Ghilezan, editors, *PPDP 2021: 23rd International Symposium on Principles and Practice of Declarative Programming, Tallinn, Estonia, September 6-8, 2021*, pages 5:1–5:11. ACM, 2021. doi:10.1145/3479394.3479400.

[15] Ugo de'Liguoro and Riccardo Treglia. From semantics to types: The case of the imperative λ-calculus. *Theor. Comput. Sci.*, 973:114082, 2023. doi:10.1016/J.TCS.2023.114082.

[16] Mariangiola Dezani-Ciancaglini, Paola Giannini, and Simona Ronchi Della Rocca. Intersection, universally quantified, and reference types. In Erich Grädel and Reinhard Kahle, editors, *18th Annual Conference of the EACSL on Computer Science Logic (CSL)*, volume 5771 of *Lecture Notes in Computer Science*, pages 209–224. Springer, 2009. doi:10.1007/978-3-642-04027-6_17.

[17] Rémi Douence and Pascal Fradet. A systematic study of functional language implementations. *ACM Transactions on Programming Languages and Systems*, 20(2):344–387, 1998. doi:10.1145/276393.276397.

[18] Thomas Ehrhard. Collapsing non-idempotent intersection types. In Patrick Cégielski and Arnaud Durand, editors, *Computer Science Logic (CSL'12) - 26th International Workshop/21st Annual Conference of the EACSL, CSL 2012, September 3-6, 2012, Fontainebleau, France*, volume 16 of *LIPIcs*, pages 259–273. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2012. doi:10.4230/LIPICS.CSL.2012.259.

[19] Thomas Ehrhard. The Scott model of linear logic is the extensional collapse of its relational model. *Theor. Comput. Sci.*, 424:20–45, 2012. doi:10.1016/J.TCS.2011.11.027.

[20] Thomas Ehrhard. Non-idempotent intersection types in logical form. In Jean Goubault-Larrecq and Barbara König, editors, *Foundations of Software Science and Computation Structures (FoSSaCS)*, volume 12077 of *LNCS*, pages 198–216, Cham, 2020. Springer International Publishing. doi:10.1007/978-3-030-45231-5_11.

[21] Thomas Ehrhard and Laurent Regnier. The differential lambda-calculus. *Theoretical Computer Science*, 309(1-3):1–41, 2003. doi:10.1016/S0304-3975(03)00392-X.

[22] Francesco Gavazzo, Riccardo Treglia, and Gabriele Vanoni. Monadic intersection types, relationally. In Stephanie Weirich, editor, *Programming Languages and Systems - 33rd European Symposium on Programming, ESOP 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6-11, 2024, Proceedings, Part I*, volume 14576 of *Lecture Notes in Computer Science*, pages 22–51. Springer, 2024. doi:10.1007/978-3-031-57262-3_2.

[23] Giulio Guerrieri, Willem B. Heijltjes, and Joseph W. N. Paulus. A deep quantitative type system. In Christel Baier and Jean Goubault-Larrecq, editors, *29th EACSL Annual Conference on Computer Science Logic, CSL*, volume 183 of *LIPIcs*, pages 24:1–24:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPIcs.CSL.2021.24.

[24] Willem Heijltjes. The Functional Machine Calculus. In *Proceedings of Mathematical Foundations of Programming Semantics (MFPS XXXVIII)*, volume 1 of *Electronic Notes in Theoretical Informatics and Computer Science*, 2022. URL: https://entics.episciences.org/10513, doi:10.46298/entics.10513.

[25] Willem Heijltjes. The Functional Machine Calculus III: Control. Submitted, 2025.

[26] Willem Heijltjes and Georgina Majury. Simple types for probabilistic termination. In Jörg Endrullis and Sylvain Schmitz, editors, *33rd EACSL Annual Conference on Computer Science Logic (CSL)*, Leibniz International Proceedings in Informatics, pages 31:1–31:22. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany, 2025. doi:10.4230/LIPIcs.CSL.2025.31.

[27] A. J. Kfoury and J. B. Wells. Principality and decidable type inference for finite-rank intersection types. In Andrew W. Appel and Alex Aiken, editors, *POPL '99, Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, San Antonio, TX, USA, January 20-22, 1999*, pages 161–174. ACM, 1999. doi:10.1145/292540.292556.

[28] Assaf J. Kfoury. A linearization of the lambda-calculus and consequences. *Journal of Logic and Computation*, 10(3):411–436, 2000. doi:10.1093/LOGCOM/10.3.411.

[29] Jan Willem Klop. *Combinatory Reduction Systems*. PhD thesis, Rijksuniversiteit Utrecht, 1980.

[30] Jean-Louis Krivine. A call-by-name lambda-calculus machine. *Higher-Order and Symbolic Computation*, 20:199–207, 2007. doi:10.1007/s10990-007-9018-9.

[31] Jim Laird, Giulio Manzonetto, Guy McCusker, and Michele Pagani. Weighted relational models of typed lambda-calculi. In *28th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 301–310, 2013. doi:10.1109/LICS.2013.36.

[32] Paul Blain Levy. *Call-by-push-value: A functional/imperative synthesis*, volume 2 of *Semantic Structures in Computation*. Springer Netherlands, 2003. doi:10.1007/978-94-007-0954-6.

[33] Damiano Mazza, Luc Pellissier, and Pierre Vial. Polyadic approximations, fibrations and intersection types. *Proceedings of the ACM on Programming Languages*, 2(POPL), 2018. doi:10.1145/3158094.

[34] Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991. doi:10.1016/0890-5401(91)90052-4.

[35] Rob Nederpelt. *Strong normalization in a typed lambda calculus with lambda structured types*. PhD thesis, Technische hogeschool Eindhoven, 1973.

[36] Peter Møller Neergaard and Harry G. Mairson. Types, potency, and idempotency: Why nonlinearity and amnesia make type systems work. In *Proc. 9th International Conference on Functional Programming (ICFP)*, pages 138–149. ACM, 2004. doi:10.1145/1016850.1016871.

[37] C.-H. Luke Ong. Quantitative semantics of the lambda calculus: Some generalisations of the relational model. In *32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, 2017. doi:10.1109/LICS.2017.8005064.

[38] Luca Paolini, Mauro Piccolo, and Simona Ronchi Della Rocca. Essential and relational models. *Math. Struct. Comput. Sci.*, 27(5):626–650, 2017. doi:10.1017/S0960129515000316.

[39] Elaine Pimentel, Simona Ronchi Della Rocca, and Luca Roversi. Intersection types from a proof-theoretic perspective. *Fundamenta Informaticae*, 121(1-4):253–274, 2012. doi:10.3233/FI-2012-778.

[40] Gordon D. Plotkin. Call-by-name, call-by-value and the λ-calculus. *Theoretical Computer Science*, 1(2):125–159, 1975. doi:10.1016/0304-3975(75)90017-1.

[41] G. Pottinger. A type assignment for the strongly normalizable λ-terms. In *To H. B. Curry: essays on combinatory logic, lambda calculus and formalism*, pages 561–577. Academic Press, London, 1980.

[42] A.J. Power and Hayo Thielecke. Closed Freyd- and κ-categories. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, volume 1644 of *Lecture Notes in Computer Science*, pages 625–634. Springer, 1999. doi:10.1007/3-540-48523-6_59.

[43] Femke van Raamsdonk, Paula Severi, Morten Heine Sørensen, and Hongwei Xi. Perpetual reductions in lambda-calculus. *Inf. Comput.*, 149(2):173–225, 1999. doi:10.1006/inco.1998.2750.